

Assistive Orchestration

A Voice-Driven, State-Aware Orchestration Framework for Distributed Assistive Smart-Environment Devices with Acknowledgement-Gated State Commitment and Idempotent Retry

PROJECT TITLE

Assistive Orchestration — voice-driven multi-device assistive environment

SUBMITTED BY

[Team member 1 — Name, ID]

[Team member 2 — Name, ID]

[Team member 3 — Name, ID]

SUPERVISOR

[Supervisor name, designation]

COURSE

[Course code and title]

DEPARTMENT

[Department, Institution]

DATE OF SUBMISSION

[dd month yyyy]

Software baseline described in this report: branch `version-3`, commit `578c205`, application version 0.10.0.

Table of Contents

1 Abstract	6
2 Introduction	7
2.1 Background of the Problem	7
2.2 Problem Statement	8
2.3 Motivation for Selecting the Problem	8
2.4 Specific Project Objectives	8
2.5 Scope of the Project	9
3 Background Study and Literature Review	11
3.1 Voice Interfaces in Assistive and Inclusive Housing	11
3.2 Local Speech Recognition on Constrained Hardware	12
3.3 Intent Recognition and Out-of-Scope Rejection	12
3.4 From Single-Device Commands to Multi-Device Orchestration	12
3.5 Reliability of MQTT Command Delivery	14
3.6 Comparative Literature Review	14
3.7 Implementation Gap Addressed by This Project	16
4 Project Contributions	18
4.1 Contributions Against Stated Objectives	18
4.2 Objectives Modified During Implementation	21
4.3 Objectives Not Completed	22
5 System Design and Methodology	24
5.1 Overall Workflow	24
5.2 System Architecture and Block Diagram	26
5.2.1 Layered Software Architecture	26
5.2.2 Hardware Block Diagram and Data/Control Flow	27
5.2.3 Environment Model and Room Topology	28
5.2.4 State Model and the Two-Tier Commit Rule	30
5.3 Hardware Interfacing and Circuit Diagrams	34
5.4 Implementation Logic and Pseudocode	35
5.5 Diagram Conventions	40
6 Experimental Setup and Engineering Design Justification	41
6.1 Components and Equipment	41
6.2 Experimental Environment	43
6.3 Measurement Instrumentation and Test Procedure	44
7 Results and Analysis	45
7.1 Final Prototype and Functional Results	45
7.2 Performance Evaluation	46
7.2.1 Intent Recognition Accuracy	46
7.2.2 Execution Latency	47
7.2.3 Idempotency Under Retry	50
7.2.4 Reliability, Containment and Recovery	52
7.2.5 Degradation of Best-Effort Execution	53
7.2.6 Software Verification	54
7.3 Analysis of Results	55
7.4 Prototype versus Real-Life Deployment	57
7.5 Challenges, Limitations and Strengths	59
8 Future Work	62

9 Conclusion	65
10 References	67
11 Appendix	69

List of Figures and Tables

Figures

Figure 1 — End-to-end workflow of the implemented system	24
Figure 2 — Layered software architecture	26
Figure 3 — Hardware block diagram with data, control and power flow	27
Figure 4 — Room topology and door assignment	28
Figure 5 — Location-certainty state machine	31
Figure 6 — Operating-mode state machine and the emergency latch	31
Figure 7 — Two-tier state commitment rule	32
Figure 8 — Command, acknowledgement and retry sequence	33
Figure 9 — Interfacing diagram, node esp32_a	34
Figure 10 — Interfacing diagram, node esp32_b	35
Figure 11 — Interfacing diagram, node esp32_c	35
Figure 12 — Final prototype photographs	45
Figure 13 — Browser dashboard during a workflow	45
Figure 14 — Reference-corpus composition	46
Figure 15 — Intent recognition outcomes and metrics	47
Figure 16 — Per-device execution latency on physical hardware	48
Figure 17 — Transport cost versus actuation cost	49
Figure 18 — Whole-workflow timing	50
Figure 19 — Response time for a replayed command	51
Figure 20 — Physical re-actuation under replay	51
Figure 21 — Reliability, retry and recovery	52
Figure 22 — Degradation of best-effort emergency execution	53

Tables

Table 1 — Comparative review of related work	15
Table 2 — Objectives mapped to delivered contributions	21
Table 3 — Supported intents and generated workflows	28
Table 4 — Action vocabulary	29
Table 5 — Room transition rules	30
Table 6 — Device registry: logical device to responsible node	30
Table 7 — Major components, specifications and selection justification	41
Table 8 — Software stack and versions	43
Table 9 — Experimental environment and operating envelope	43
Table 10 — Intent recognition results on disjoint evaluation sets	47
Table 11 — Measured per-device execution latency	48
Table 12 — Replay response time and re-execution count	50
Table 13 — Reliability under node-availability conditions	52
Table 14 — Emergency workflow duration	53
Table 15 — Automated test suite composition	54
Table 16 — Objective achievement summary	56
Table 17 — Functional requirements and verification status	57
Table 18 — Prototype configuration versus deployment configuration	59

Table 19 — Firmware and host constants governing measured behaviour	70
Table 20 — Composition of the evaluation datasets	70

1 Abstract

Assistive smart environments are intended to give people with limited mobility independent control of their surroundings, yet most deployed voice-controlled systems still treat a spoken request as a single device command. Activities that matter to a resident — settling down to study, preparing for sleep, leaving the house, evacuating during an emergency — are not single operations. They are ordered sequences of door, lighting and appliance actions whose content depends on where the person currently is and on what is already switched on. A further difficulty is that low-cost actuators give no feedback, so a controller that assumes a published command succeeded will silently accumulate a false picture of the home.

The objective of this project was to design and build a voice-driven orchestration framework in which a high-level spoken intent is interpreted locally, expanded into a state-dependent sequence of device actions, dispatched across distributed microcontroller nodes, and — critically — committed to the system's logical model of the environment only when the responsible node has acknowledged the action. The implemented solution captures speech at 16 kHz, transcribes it offline with faster-whisper, and matches the transcript against a 313-sentence reference corpus using `all-MiniLM-L6-v2` sentence embeddings and maximum cosine similarity against a rejection threshold of 0.65. A recognised intent is executed only after explicit human confirmation. An orchestrator then applies two policy gates, generates an ordered action list from the current environment state, and dispatches each action over MQTT at QoS 1 to one of three ESP32 nodes driving five LEDs, ten SG90 servos and one buzzer. A retry reuses the original command identifier and each node deduplicates against a 32-entry ring buffer, so a lost acknowledgement cannot actuate a door twice. Environment state is persisted atomically and restored under an explicit crash-recovery policy.

The principal contribution is a measured demonstration that acknowledgement-gated commitment with identifier-stable retry eliminates duplicate physical actuation on feedback-less hardware: across forty replayed commands on ten devices, every replay was answered from stored node state with zero re-actuations, whereas the same replays with the deduplication mechanism compiled out re-executed four of four times on every servo tested. Recognition on a disjoint 110-sentence holdout set reached 0.718 accuracy with a weighted precision of 1.00, meaning every error was a refusal rather than a wrong action. The prototype establishes that reliable multi-device assistive orchestration is achievable on commodity microcontrollers without any sensing, provided that state commitment is made conditional on correlated device replies.

Keywords — assistive environment; voice user interface; semantic intent recognition; out-of-scope rejection; multi-device orchestration; MQTT; ESP32; idempotent actuation; state consistency; Internet of Things.

2 Introduction

2.1 Background of the Problem

A person with restricted mobility interacts with a home through a small number of repeated environmental operations: opening and closing doors between rooms, switching lights on and off in the room being entered and the room being left, preparing a table or a bed, operating an appliance, and raising an alarm. For a resident who cannot easily cross a room to reach a switch, each of these operations represents a dependency on another person. Assistive smart-environment technology exists precisely to remove that dependency, and voice is the most natural interface for it because it requires no reach, no grip and no fine motor control.

Commercial and academic voice-controlled home automation has made the single-device case routine. A spoken phrase is transcribed, matched to a device and an operation, and a command is transmitted. What remains difficult is the case where the request describes an *activity* rather than an operation. "Prepare everything for sleep" is not a switch; it is a claim about the desired end state of an entire environment, and satisfying it may require switching off the appliance in the room being vacated, opening the door of that room, closing it again, lighting the corridor, opening the bedroom door, closing it, lighting the bedroom, preparing the bed, and then extinguishing the corridor light. Which of those operations are actually required depends entirely on where the person is standing when the request is made and on which devices are already in the desired condition.

A second difficulty is specific to low-cost assistive hardware. A hobby-grade servo that moves a door panel, a relay that switches a lamp, and a buzzer that raises an alarm all share a property: none of them reports back. The controller can publish a command and the transport can confirm that the broker accepted the packet, but nothing in that chain establishes that a door actually opened. If the controller records the intended effect regardless, its internal model of the home diverges from the home. Once that divergence exists, every subsequent decision — which door to open next, whether a corridor light is already on, whether the person is in the room the system believes — is computed from a false premise. In an assistive context the consequence is not an inconvenience but a resident trapped behind a door the system believes to be open.

2.2 Problem Statement

PROBLEM STATEMENT

Existing voice-controlled smart environments interpret user speech as isolated device commands and commit the intended effect of those commands optimistically, without confirmation from the device. Assistive activities, however, require an ordered and context-dependent sequence of operations across several devices and several controllers, and the actuators available at prototype cost provide no feedback from which the controller can determine whether an operation actually took place. The problem addressed by this project is therefore the design and implementation of a voice-driven orchestration layer that converts a natural-language activity request into a state-dependent multi-device action sequence, dispatches that sequence across distributed nodes, correlates each dispatched action with the reply it produces, admits an action's effect into the system's model of the environment only when that correlation succeeds, and retries a lost action in a manner that cannot cause the same physical operation to be performed twice.

2.3 Motivation for Selecting the Problem

Three considerations motivated the selection of this problem.

The engineering difficulty lies above the recognition layer. Offline speech recognition and sentence-embedding similarity are both available as mature, pre-trained components that require no training data of the kind an undergraduate project can realistically collect. Assembling them is not, in itself, an engineering contribution. The unsolved part of the problem — deciding what a request means *given the current state of the environment*, and keeping that state honest in the presence of silent hardware — sits entirely in the layer above, and that layer is where the project's effort was directed.

Silent failure is the dominant failure mode of cheap actuation. During data collection for this project an incident occurred that illustrates the point precisely: six servos lost their power supply, yet every one of them returned a success status, because the firmware's completion reply is emitted after a fixed actuation delay rather than after any confirmation of movement. A design that could not distinguish this case from a successful one would have recorded a prepared bed that did not exist. Making the boundary between "the node answered" and "the device moved" explicit, measurable and honestly reported became a central objective rather than an afterthought.

Retry and safety are in direct tension. Any system that tolerates an unreliable wireless link must retry. Any system that drives a physical door must not perform the same movement twice because an acknowledgement was lost on the return path. Resolving that tension requires a deliberate identifier scheme and node-side duplicate suppression, and it is a resolution that can be tested and quantified on real hardware — which makes it a suitable subject for an experimental undergraduate project rather than a purely descriptive one.

2.4 Specific Project Objectives

The project was undertaken against eight specific objectives. These objectives are referenced throughout the report and are formally evaluated against the delivered system in Section 4 and again in Section 7.3.

01 — Natural-language voice input with local processing

Accept spoken requests phrased as activities rather than device names, and perform speech capture and transcription entirely on the local machine, without transmitting audio to any external service.

02 — Semantic intent recognition with explicit rejection

Map a transcript to one of a fixed set of supported intents by semantic similarity rather than keyword matching, and refuse any request whose similarity falls below a defined threshold instead of executing the closest available guess.

03 — Context and state representation

Maintain a logical model of the environment covering the occupied room, the operating mode, the certainty of the location, and the condition of every controllable device, and make that model the input to interpretation rather than a by-product of it.

04 — State-dependent workflow generation

Expand a single accepted intent into an ordered sequence of device-level actions computed at the moment of execution from the current state, emitting only the actions that are still required.

05 — Distributed multi-node execution

Address logical devices independently of the controller that owns them, resolve each device to a responsible node through a registry, and dispatch commands to several physically separate microcontrollers over a common message bus.

06 — Acknowledgement-gated state commitment

Admit the logical effect of an action into the environment model only after a reply has been received, correlated to the dispatched command, and validated as successful for the device and operation concerned.

07 — Reliable and idempotent recovery from communication failure

Retry an unacknowledged action within a bounded attempt budget in a manner that cannot cause the corresponding physical operation to be performed more than once, and report a permanent failure when the budget is exhausted.

08 — Durable state with safe restart behaviour

Persist the environment model across process restarts, detect an unclean shutdown, and adopt a recovery posture that refuses location-dependent commands rather than acting on a location that can no longer be trusted.

2.5 Scope of the Project

The delivered system is a laboratory prototype. Its scope is bounded as follows.

Included. A fixed environment model of six locations — an outside position, a drawing room acting as the sole corridor, and four functional rooms for study, relaxation, sleep and meals. Sixteen logical devices distributed across three ESP32 nodes. Eleven supported intents, each with a workflow builder that reads current state. Offline speech transcription and embedding-based intent recognition. MQTT transport at QoS 1 with a five-second acknowledgement window and up to two retries. Atomic state persistence with an append-only audit log. A browser dashboard with a live event stream, and an equivalent console interface.

Excluded by design. The prototype contains no sensor of any kind. It cannot detect where the resident actually is, whether a door physically moved, or whether a room is occupied;

the resident's location is established by declaration and maintained by inference from completed workflows. The system performs no autonomous navigation and does not control a wheelchair or any mobility aid. Workflow generation is deterministic and rule-based; no generative planner produces action sequences at run time. The medication mechanism is an actuation demonstrator and makes no claim regarding dosage, scheduling or clinical suitability. Security mechanisms are absent, and the system is intended to be operated only on an isolated laboratory network. No user study with members of the target population was conducted, so no claim is made about usability or acceptability.

Evaluation scope. All quantitative results reported in Section 7 were obtained either from the physical three-node prototype on a local network or from a deterministic automated test suite. Where a figure derives from software simulation rather than hardware, it is identified as such at the point of use. Quantities that were not measured — notably end-to-end latency from the start of an utterance to the first actuation, and power consumption — are listed explicitly in Section 7.5 rather than estimated.

3 Background Study and Literature Review

Sixteen works were reviewed and are grouped below into five themes that correspond to the layers of the implemented system: voice interaction in assistive housing, local speech recognition, intent recognition with rejection, multi-device orchestration, and the reliability of the underlying message transport. Twelve of these works are compared directly in Table 1. The majority were published between 2023 and 2026; four older works are retained because they define the techniques the implementation depends on.

3.1 Voice Interfaces in Assistive and Inclusive Housing

Vigouroux *et al.* [1] report the most directly relevant field evidence. Working within the HIP HOPE inclusive-housing project, they observed seven participants with different disabilities operating home-automation equipment through a commercial voice assistant, and recorded command success rates by type of impairment. Their failure analysis is the useful part for a designer: recognition failures clustered around command formulation, noisy environments, intelligibility of speech, speech segmentation and poor synchronisation of the audio channel opening. The study is an evaluation of an existing cloud assistant rather than a system contribution, and it stops at the level of individual equipment commands, but it establishes that in this population the interface fails for reasons that additional device intelligence cannot repair — which argues for a design that refuses uncertain input rather than guessing at it.

Irugalbandara *et al.* [2] address a different weakness of the same commercial stack. Observing that voice-assisted home automation has been of particular benefit to elderly and disabled users but that dependence on cloud services creates exposure to attack and fails where connectivity is poor, they implement a home automation system with speech recognition and power measurement that operates independently of the internet. The relevance here is the design premise rather than the feature set: local processing is treated as a structural property of an assistive system, not an optimisation. Their contribution remains device-level control with monitoring; no notion of a multi-step activity appears.

Brunete *et al.* [3] go furthest toward integration. Their architecture combines IoT devices, a service robot and a multimodal interface within a smart room aimed at people with disabilities and bedbound patients, and it is demonstrated on a physical prototype including an assistive robotic arm. This is genuine multi-device coordination, and the inclusion of robotics makes it considerably more capable physically than the present work. It is, however, an integration architecture: the coordination logic is not presented as a state-conditioned planning problem, and the treatment of device-reply verification is not the focus.

Katumba *et al.* [4] are the closest structural analogue to the hardware arrangement used in this project. They build a Luganda speech-intent classification system for IoT, with a Raspberry Pi performing voice processing, a Wio Terminal as display, and ESP32 nodes controlling the devices, and they release a curated command dataset. Their recogniser uses MFCC acoustic features with a two-dimensional convolutional network — that is, classification directly from audio rather than from a transcript. The split of responsibility between a capable host and thin ESP32 actuator nodes is identical in spirit to the arrangement adopted here; the difference is that their intent maps to a device operation, whereas in this work an intent maps to a sequence whose content is computed from state.

3.2 Local Speech Recognition on Constrained Hardware

Radford *et al.* [20] introduced Whisper, an encoder-decoder transformer trained by large-scale weak supervision, which supplies the transcription capability used in this project through the `faster-whisper` reimplementation. The relevant question for a prototype is not accuracy in the abstract but whether the model is deployable without a cloud dependency. Dutta *et al.* [5] answer that question for constrained hardware: motivated explicitly by privacy and regulatory concerns around cloud ASR inference, they adapt Whisper for on-device use and demonstrate a fine-tuned `tiny.en` model running on a Raspberry Pi, reporting a word error rate of 15.9 per cent on the MyST corpus and real-time factors between 0.23 and 0.41, while noting thermal throttling when larger variants are attempted. Their finding supports the decision taken here to run the `base` model on a host CPU with int8 quantisation rather than to attempt inference at the node, and it also indicates the direction any future migration of transcription to the edge would have to take.

3.3 Intent Recognition and Out-of-Scope Rejection

Reimers and Gurevych [6] established the technique on which the recognition layer of this project rests: a siamese-network modification of BERT that produces sentence embeddings whose cosine similarity is a meaningful measure of semantic closeness, making similarity search over a reference corpus computationally practical. The embedding model used here, `all-MiniLM-L6-v2`, is a compact member of that family.

Larson *et al.* [7] made the complementary argument that an assistant's ability to recognise in-scope intents is not a complete description of its behaviour, and introduced an evaluation dataset designed specifically around out-of-scope prediction. Their framing — that a deployed assistant will receive requests it was never designed to serve, and that its response to those requests is a first-class evaluation target — is the direct justification for reporting rejection rate alongside accuracy in Section 7.2.1 rather than reporting accuracy alone.

Zhang *et al.* [8] extend this line to the specific configuration used here. They note that similarity of embeddings from transformer-based sentence encoders is one of the most accurate approaches to out-of-scope rejection, and that a motivating deployment is the hybrid on-device assistant, where a compact local model must decide whether a request is within its competence before either serving it or escalating it. Their own contribution is a fine-tuning regulariser that tightens the in-scope embedding distribution, reporting an improvement of one to four percentage points in the area under the precision-recall curve for rejection. The present project deliberately does not fine-tune — the encoder is frozen and the reference corpus is the only domain adaptation — which places its recognition performance at the untuned end of this spectrum and identifies fine-tuning as a concrete, evidence-backed avenue for improvement.

3.4 From Single-Device Commands to Multi-Device Orchestration

King *et al.* [9] state the orchestration problem in almost the same terms adopted in Section 2.1. They observe that assistants perform well on direct, well-specified commands or on hard-coded routines, but that natural speech frequently describes a goal — their examples include making a room cosy or saving energy — rather than naming a device and an action, and that current systems cannot reason about devices and settings in relation to human situations. Their system, Sasha, uses a large language model to respond to such

loosely constrained commands, and their user study finds that the model reasons creatively but exhibits recurring failure patterns that reduce its usefulness. The problem statement is shared; the proposed remedy differs fundamentally, since the present work replaces open-ended generation with deterministic builders.

Rivkin *et al.* [10] address the grounding problem that Sasha exposes. SAGE triggers an LLM-controlled sequence of discrete actions that can retrieve information, interact with the user or manipulate device states, and controls that sequence through a dynamically constructed tree of prompts that decides the next action, judges whether an action succeeded, and determines when to stop. On a benchmark of fifty smart-home tasks it achieves a 75 per cent success rate against 30 per cent for LLM-enabled baselines. Two observations follow. First, the decomposition of a request into a controlled sequence of discrete, verifiable actions is the same architectural move made here. Second, a 75 per cent success rate is an entirely respectable research result and an unacceptable one for a door that a resident depends on, which is the reason this project constrains the action sequence to deterministic generation.

Yu *et al.* [11] quantify why. Their motivation for IoTGPT is that existing LLM-based approaches suffer from unreliable and inefficient device control owing to the non-deterministic nature of the models, high inference latency and cost, and limited personalisation. Their remedy is to decompose instructions into subtasks, memorise them, and reuse learned subtasks so that later instructions require fewer model calls. This is, in effect, a mechanism for converting non-deterministic planning into cached deterministic plans — a convergence toward the fixed builders used in this project, arrived at from the opposite direction.

Zhan *et al.* [12] present the work most closely aligned with the reliability concerns of this project. HearthNet argues that smart-home control differs from session-scoped agent delegation because it is persistent, event-driven, failure-prone and tied to physical devices with no shared context window, and that real deployments are brittle because devices fail and recovery requires manual intervention. Their system deploys persistent role-specialised agents at an edge hub that coordinate through MQTT, Git-backed shared state and root-issued actuation leases, separating planning, verification, authorisation and actuation across explicit boundaries, and they demonstrate rejection of stale or unauthorised commands before actuation. The problem framing is shared almost exactly, and the separation of verification from actuation is the same principle applied here. The differences are that HearthNet places LLM agents at the planning layer and uses hosted inference, whereas this project's planning layer is deterministic and fully local; and that HearthNet reports live scenario demonstrations rather than repeated-trial measurements of duplicate actuation on feedback-less hardware, which is the specific quantity measured in Section 7.2.3.

Du *et al.* [13] scale the problem outward. LLMind 2.0 distributes lightweight agents to the devices themselves and uses natural language as the machine-to-machine medium, with a central coordinator issuing subtask descriptions that device agents translate into local code against their own APIs. It contributes a timeout-based deadlock-avoidance protocol for coordinating distributed subtask execution, retrieval-augmented subtask-to-API mapping, and fine-tuned lightweight models for reliable code generation. The timeout-based coordination of distributed subtasks is the same class of mechanism as the acknowledgement window used here, arrived at for the same reason: a distributed executor cannot wait indefinitely for a node that may never answer.

Cheng *et al.* [14] approach reliability through verification instead. AutoIoT generates trigger-action automation rules from device information using an LLM and then applies formal conflict detection and avoidance, on the premise that users often lack the knowledge to formulate rules and that platform-supplied policies are too simple and may conflict. Their insight that conflicts between automation rules are a first-class hazard supports a design decision taken here for a different reason: serialising workflows under a single execution lock removes the possibility of two activity sequences interleaving against one physical environment.

Seo *et al.* [15] supply the evaluation critique. SimuHome provides a Matter-grounded simulator and a 600-episode benchmark, on the argument that existing smart-home benchmarks treat the home as static, neither simulating how device operations affect environmental variables over time nor supporting workflow scheduling of device commands. Across eighteen evaluated agents they find workflow scheduling to be the hardest category, with failures persisting across alternative frameworks and fine-tuning, and they propose time-accelerated simulation as a place for agents to pre-validate actions before committing them to the real world. Two conclusions bear on this report: the ordered scheduling of device commands is independently confirmed as the difficult part of the problem rather than the easy part, and the distinction between validating in simulation and committing in the real world is exactly the distinction this report maintains when separating simulated from hardware-measured results.

3.5 Reliability of MQTT Command Delivery

The MQTT specification [17] defines three delivery qualities: at most once, at least once, and exactly once. QoS 1 — at least once — guarantees delivery but explicitly permits duplicates, and QoS 2 achieves exactly-once semantics through a four-step handshake at greater cost. Critically, all three levels describe the hop between a client and its broker. None of them describes whether the subscribing node processed the message or whether a physical actuator moved, which is why application-level correlation was required in this project rather than a higher QoS setting.

Ali *et al.* [16] study the reliability of the protocol itself under adverse conditions, proposing and validating a retransmission-based variant intended to improve quality of service where networks are unreliable and where data integrity is routinely traded away to conserve bandwidth. Their work is a transport-layer treatment: it improves the probability that a message arrives. It does not, and is not intended to, address the consequence of that arrival being duplicated at an actuator, which is the hazard the deduplication mechanism reported in Section 7.2.3 was built to eliminate.

3.6 Comparative Literature Review

Table 1. Comparative review of related work. "Ack-gated commit" indicates whether the reported system admits a device action into its own persistent model of the environment only after a correlated, validated reply from the device.

Reference	Proposed system	Method / technology	Hardware / components	Major capability	Limitation
[1] Vigouroux <i>et al.</i> , 2023	Field study of voice control in inclusive housing	Commercial cloud voice assistant; speech-error analysis by impairment type	Off-the-shelf smart speaker; installed home-automation equipment	Empirical command success rates for seven disabled participants; categorised failure modes	Evaluation only, no system contribution; single-equipment commands; cloud dependent
[2] Irugalbandara <i>et al.</i> , 2023	Offline secure home automation with power measurement	Local speech recognition; no cloud services	Embedded controller, appliance switching, power sensing	Operates without internet; removes cloud attack surface; measures consumption	Device-level control only; no activity-level sequencing or state reasoning
[3] Brunete <i>et al.</i> , 2021	Smart assistive architecture integrating IoT and robotics	Multimodal interaction architecture for healthcare environments	Smart room, home-automation devices, assistive robotic arm	Genuine multi-device and robot coordination for bedbound users; physical prototype	Integration architecture; device-reply verification not the focus; costly hardware
[4] Katumba <i>et al.</i> , 2024	Luganda speech-intent recognition for IoT	MFCC features with a 2-D convolutional network; classification from audio	Raspberry Pi (processing), Wio Terminal (display), ESP32 nodes (actuation)	Low-resource-language voice control; open-sourced command dataset; host/node split	Intent maps to one device operation; no environment state; no acknowledgement gating
[5] Dutta <i>et al.</i> , 2025	Lightweight Whisper ASR for edge devices	Fine-tuned <i>tiny.en</i> ; low-rank compression; filtering strategies	Raspberry Pi; CPU and GPU inference profiling	15.9 % WER on MyST; real-time factor 0.23-0.41; quantified thermal limits	Transcription only; no intent, orchestration or actuation layer
[8] Zhang <i>et al.</i> , 2024	Fine-tuning sentence encoders for intent and out-of-scope detection	Cross-entropy regularised by an autoencoder reconstruction loss	Software only; on-device assistant scenario	1-4 % gain in rejection AUPR without loss of in-scope accuracy	Requires labelled fine-tuning data; no device control component
[9] King <i>et al.</i> , 2023	Sasha — goal-oriented reasoning for under-specified commands	LLM planning from loosely constrained goals; age-diverse user study	Simulated and connected smart-home devices	Creative achievement of goals such as "make it cozy"; empirical failure taxonomy	Recurring generative failure patterns reduce usefulness; non-deterministic plans
[10] Rivkin <i>et al.</i> , 2023	SAGE — smart home agent with grounded execution	LLM-controlled tree of discrete actions; self-judged success and termination	Real smart devices accessed through APIs; room photographs	75 % success on 50 hard tasks vs 30 % baseline; device-agnostic API reading	Residual 25 % failure unacceptable for safety-relevant actuation; cloud inference
[11] Yu <i>et al.</i> , 2026	IoTGPT — reliable and personalised LLM smart-home agent	Instruction decomposition into memorised, reusable subtasks	Commercial smart-home device set	Fewer model calls per instruction; reduced latency and cost; per-subtask personalisation	Reliability obtained by caching around non-determinism, not by removing it
[12] Zhan <i>et al.</i> , 2026	HearthNet — edge multi-agent orchestration	Persistent role-specialised agents; MQTT; Git-backed state; actuation leases	Commodity edge hardware and Android devices; heterogeneous devices via adapters	Separates planning, verification, authorisation and actuation; rejects stale commands	Hosted LLM inference; live scenario demonstrations rather than repeated-trial measurement

Reference	Proposed system	Method / technology	Hardware / components	Major capability	Limitation
[13] Du et al., 2025	LLMind 2.0 — distributed IoT automation	Natural-language M2M; RAG subtask-to-API mapping; timeout dead-lock avoidance	Multi-robot warehouse and Wi-Fi deployment testbeds	Scales across heterogeneous devices; distributed local code generation	Per-device model hosting; domain is industrial rather than assistive
[14] Cheng et al., 2024	AutoloT — automated trigger-action platform	LLM rule generation plus formal conflict detection and avoidance	Integrates with existing platforms (HomeKit, SmartThings class)	Conflict-free rule generation; code adapter separates reasoning from syntax	Static rules, not state-conditioned sequences; no execution-time verification
[15] Seo et al., 2025	SimuHome — temporal and environment-aware benchmark	Matter-grounded simulator; 600 episodes; time-accelerated evaluation	Simulation only	Shows workflow scheduling is the hardest category across 18 agents	No physical actuation; findings do not transfer automatically to real hardware
[16] Ali et al., 2022	Retransmission-based mutant MQTT	Mathematical modelling and validation of a QoS-improving protocol variant	Smart-city messaging infrastructure	Improved delivery reliability on unreliable networks without excess bandwidth	Transport layer only; duplicate delivery at the actuator remains unaddressed
This work	Assistive Orchestration	Frozen sentence embeddings with threshold rejection; deterministic state-conditioned workflow builders; correlated acknowledgement gating; identifier-stable retry with node-side deduplication	Host PC, Mosquitto broker, 3 × ESP32-WROOM-32, 5 LEDs, 10 SG90 servos, 1 buzzer	One intent expands to a state-dependent multi-device sequence; state commits only on validated reply; 40/40 replays served without re-actuation; precision 1.00 on holdout	No sensors; strictly sequential execution; fixed environment model; no security; no user study

3.7 Implementation Gap Addressed by This Project

Read together, the reviewed works leave a specific gap, and it is narrower than "voice-controlled smart homes have not been built".

The assistive voice systems verify the user but not the device. Works [1]–[4] are concerned with whether the system understood the person. Their evaluation quantities are recognition success rates and command accuracy. None of them reports what the controller does when a device is addressed and does not answer, because in each case the command is a single operation whose failure is immediately visible to the user standing in the room. That assumption does not survive the move to multi-step activities, where the fifth action in a sequence may fail while the user has no way of knowing that the first four were premised on it.

The orchestration systems verify the plan but not the actuator. Works [9]–[15] take the activity-level request seriously and produce genuine multi-device sequences. Their verification effort, however, is directed at the plan: whether the generated action sequence is correct, conflict-free, or scheduled in a feasible order. SAGE judges whether an action suc-

ceeded from within the language model's own reasoning loop [10]; AutoIoT verifies rules formally before they ever run [14]; SimuHome validates in an accelerated simulation precisely so that commitment to the real world can be deferred [15]. HearthNet comes closest to the concern of this project by separating verification from actuation and rejecting stale commands before they reach a device [12], but its demonstration is scenario-based. In every case the physical device is assumed to be a reliable executor of a correctly planned action.

The transport work stops at the broker. QoS 1 guarantees delivery and tolerates duplicates by design [17]; work such as [16] improves the delivery probability further. Neither addresses what a duplicate does when the subscriber is a servo attached to a door.

The gap, stated precisely, is the absence of a system that *combines* activity-level intent expansion with per-action, correlated verification against feedback-less physical actuators, and that reports measured evidence about what happens at that boundary. This project addresses that gap as follows.

- It makes the logical effect of every action conditional on a reply that is correlated by command identifier and validated against the device and operation that were requested, so that a plan which is correct on paper cannot by itself change the system's model of the home.
- It separates the commitment of device state from the commitment of the resident's room and the operating mode, so that a partially executed activity leaves an honestly partial record rather than a confident wrong one.
- It resolves the retry-versus-duplicate-actuation tension explicitly, by holding the command identifier stable across attempts and suppressing duplicates at the node, and it *measures* the result against a build with the mechanism removed — a controlled comparison that the reviewed literature does not report for physical assistive actuators.
- It quantifies the cost of its own design choices rather than only their benefits, including the 13.8-fold latency penalty that strictly sequential best-effort execution imposes on an emergency workflow when devices are unreachable (Section 7.2.5).
- It keeps the entire interpretation path local and deterministic, which trades the open-ended flexibility demonstrated in [9] and [10] for behaviour that is reproducible under test — an appropriate trade for a system whose outputs are doors.

4 Project Contributions

This section states what the implemented system delivers against each objective defined in Section 2.4, and identifies the objectives whose form changed during implementation together with the technical reason for the change. Every claim below is supported either by a measurement reported in Section 7 or by a test in the automated suite, and the supporting evidence is named alongside the claim.

4.1 Contributions Against Stated Objectives

O1 — Natural-language voice input with local processing

- A browser interface and an equivalent console interface accept spoken requests phrased as activities. Audio is captured at 16 kHz, single channel, 32-bit float through the host sound device.
- Transcription is performed on the local machine by faster-whisper using the `base` model on CPU with int8 quantisation. No audio and no transcript is transmitted to any external service at any point in the pipeline.
- Empty recordings, empty transcripts, microphone access failure and concurrent recording attempts are each detected and reported as distinct conditions rather than being allowed to propagate into the recognition stage.

O2 — Semantic intent recognition with explicit rejection

- Eleven intents are supported. A transcript is embedded and compared by cosine similarity against all 313 sentences of the reference corpus; an intent's score is the maximum similarity over its own reference sentences, not a centroid, so a single well-matched phrasing is sufficient.
- A transcript scoring below the threshold of 0.65 yields the decision `UNKNOWN` and no workflow is generated. The decision vocabulary is exactly `{PREDICTED, UNKNOWN}` and this is enforced by an automated contract test.
- Measured on a disjoint 110-sentence holdout set, weighted precision is 1.00 for every one of the eleven intents: all thirty-one errors were refusals and none was a substitution of one intent for another (Section 7.2.1).
- The similarity threshold is defined once, in the intent package, so that the application, the recogniser and the test suite cannot drift apart.

O3 — Context and state representation

- An explicit environment model holds the current room, the operating mode, the return target, the location-certainty status with its destination and reason, emergency declaration and clearance timestamps, and the condition of all sixteen devices grouped by room.
- Location certainty is modelled as a separate axis with three values — `KNOWN`, `IN_TRANSIT`, `UNKNOWN` — deliberately not as a member of the room enumeration, because rooms are used as keys in the transition tables and an "unknown room" would be silently unroutable instead of loudly refused.
- A single component owns all mutation of this model. No other component writes to it.

O4 — State-dependent workflow generation

- Each of the eleven intents has a builder that reads current state at the moment of execution and emits only the actions still required. A light already on produces no action; a door already open is not opened again.
- Room movement is generated rather than enumerated: a transition builder derives the door and lighting sequence for any origin-destination pair from the topology, enforcing that every functional-to-functional move is routed through the corridor and that every door opening is immediately paired with a closing.
- The expansion factor is substantial and measurable: the shutdown intent generates eleven actions from the drawing room, and an emergency generates up to six across three nodes.
- Transition invariants and workflow-map integrity are covered by fourteen and two automated tests respectively.

O5 — Distributed multi-node execution

- Sixteen logical device identifiers are resolved to three ESP32 nodes through a device registry, so a workflow builder never refers to a node and a node never appears in the environment model.
- One firmware sketch serves all three nodes, specialised at compile time. A single workflow routinely spans two or three nodes.
- An automated test asserts that every action type in the vocabulary is handled by some node, closing the gap between what the orchestrator can emit and what the field layer can execute.

O6 — Acknowledgement-gated state commitment

- A reply is accepted only if it carries the command identifier that was dispatched, names the same device and the same action, and reports success. A reply that fails any of these checks produces no state change.
- Commitment is two-tier. A device field is committed as each acknowledgement arrives; the resident's room, the operating mode and the return target are committed only after the whole sequence has run. A half-executed activity therefore leaves a record that is partial and correct rather than complete and wrong.
- The emergency case inverts the rule deliberately in one direction and tightens it in the other: the emergency mode is latched *before* any hardware is addressed, so a partly failed alarm is still an emergency, whereas the emergency is cleared only if every action in the clearing sequence was acknowledged.
- Thirty-two automated tests cover the emergency lifecycle and thirteen cover commitment-gating regressions specifically.

O7 — Reliable and idempotent recovery from communication failure

- An unacknowledged action is retried within a five-second window for up to three total attempts. A node that replies with an explicit error is *not* retried, because the failure is a decision rather than a loss.
- The command identifier is held stable across all attempts of one logical action, and each node maintains a 32-entry ring buffer of completed identifiers. A replayed identifier is answered from stored state, flagged as a duplicate, and does not reach the actuator.

- This is the project's principal measured contribution. Across forty replays spanning all ten instrumented devices, every replay was served from stored state with zero re-executions; with the deduplication mechanism compiled out, every servo tested re-executed on four of four replays (Section 7.2.3).
- A measurable side effect follows from the firmware replying only after its actuation sequence: a deduplicated replay of the medication actuator costs 27 ms against 1 915 ms for a genuine execution, a separation of roughly seventy-two times that makes re-execution detectable by timing alone on slow actuators.
- Failure containment is exact under the fail-fast policy: after an abort, none of the thirty remaining planned actions across the trial set was dispatched.

O8 — Durable state with safe restart behaviour

- State is written atomically after every workflow and on clean shutdown, with a backup copy retained. Restoration is all-or-nothing: any invalid field causes the system to record the rejection reason and boot from a fresh model rather than resume a partially valid environment.
- An append-only audit log records one line per acknowledged and committed action, carrying the timestamp, intent, device, action, command identifier, node, status and the resulting state.
- A recovery policy distinguishes a clean shutdown from a crash. After an unclean shutdown the stored location is downgraded to `UNKNOWN` with a human-readable reason, and location-dependent intents are refused until the resident declares where they are. A first boot, detected by comparing the restored model against a fresh one, is exempted so that a new user is not locked out.
- Actuators are never replayed from the audit log. This is what keeps the single non-idempotent physical action in the system — medication activation — safe across a restart. Thirty-six automated tests cover startup and recovery behaviour.

Table 2. Objectives mapped to delivered contributions and to the evidence supporting each. "Suite" refers to the automated test suite described in Section 7.2.6.

Obj.	Delivered capability	Status	Primary evidence	Residual limitation
O1	Offline capture and transcription; activity-level phrasing accepted	Achieved	Functional demonstration, Section 7.1; no network egress in the speech path	Transcription latency never instrumented; single speaker, English only
O2	11 intents; embedding similarity with threshold rejection	Achieved	Table 10; 0.718 accuracy and weighted precision 1.00 on hold-out	Accuracy limited by a small untuned reference corpus
O3	Environment model with a separate three-valued certainty axis	Achieved	Suite: 23 location-semantics tests	Certainty derives from declaration and inference, never from sensing
O4	State-conditioned builders; generated room transitions	Achieved	Suite: 14 transition-invariant tests; 11-action shutdown measured	Builders are hand-written methods; adding an intent requires code
O5	16 devices over 3 nodes through a registry	Achieved	Table 6; suite: node action-coverage and executor-parity tests	Execution is strictly sequential; maximum concurrency is one
O6	Correlated reply validation; two-tier commit	Achieved	Suite: 13 commit-regression and 32 emergency-lifecycle tests	An acknowledgement proves the node acted, not that the device moved
O7	Identifier-stable retry with node-side deduplication	Achieved	Table 12; 40/40 replays without re-actuation, 4/4 with the mechanism removed	Timeout and retry budget are compile-time constants
O8	Atomic snapshot, audit log, crash-aware recovery policy	Achieved	Suite: 36 startup-recovery and 26 persistence tests	Audit log has no rotation and embeds a full state snapshot per line

4.2 Objectives Modified During Implementation

Two objectives were realised in a different form from the one originally proposed. Both changes were made for technical reasons and both are reflected in the system as delivered.

M1 — Ambiguity handling replaced by threshold rejection with mandatory confirmation

Originally proposed. A three-valued recognition decision — `ACCEPTED`, `AMBIGUOUS`, `UNKNOWN` — in which a confidence margin between the best and second-best intent would place borderline utterances into an ambiguous band, triggering up to two spoken clarification exchanges before the request was cancelled.

As implemented. A two-valued decision, `PREDICTED` or `UNKNOWN`, governed by a single similarity threshold, followed by a mandatory human confirmation step that the resident must complete before any workflow is dispatched. On the browser path the confirmation is enforced by a pending-intent lock: the confirmation request is refused unless a pending intent exists and matches the one being confirmed. On the console path it is an explicit prompt.

Technical justification. The objective behind the original design was that no uncertain request should actuate hardware. The implemented design satisfies that objective more directly and with fewer moving parts, for four reasons. First, the margin-based band introduces a second tunable parameter whose correct value cannot be established from a corpus of 313 reference sentences without over-fitting to it; the single threshold has one parameter, defined in one place, and is covered by a contract test. Second, a clarification dialogue requires a multi-turn conversational state machine to be maintained alongside the execution state machine, and the interaction between a pending clarification and a latched

emergency is a source of failure modes that carry no corresponding safety benefit. Third, the confirmation step is strictly stronger than clarification as a safety gate: clarification is invoked only for utterances the system itself judges to be borderline, whereas confirmation is required for *every* recognised intent, including those recognised with high confidence but transcribed incorrectly — a failure mode that Vigouroux *et al.* [1] identify as common in precisely this user population and that a similarity margin cannot detect, because the margin measures agreement between transcript and corpus, not between speech and transcript. Fourth, the measured outcome supports the choice: on the holdout set weighted precision is 1.00 for all eleven intents, so the system never substituted one activity for another, and every recognition error surfaced as a refusal that the resident can simply repeat.

M2 — Node count increased from two to three, with device ownership reorganised by room

Originally proposed. Two ESP32 nodes.

As implemented. Three nodes, partitioned by room so that each node owns a contiguous region of the environment: `esp32_a` serves the drawing room and relax room together with the house exit and the alarm, `esp32_b` serves the sleep room, and `esp32_c` serves the study and meal rooms.

Technical justification. The constraint is pin count and simultaneous servo load, not software architecture. Sixteen devices comprising ten servos, five lights and one buzzer cannot be driven from two boards without exhausting usable general-purpose pins once the reserved pins of the module variant are excluded, and concentrating more servos on one board increases the peak current drawn through a single supply path. Partitioning by room also yields a property that is useful for the reliability experiments reported in Section 7.2.4: because a room's devices are owned by one node, the loss of a node is equivalent to the loss of a region of the home, which is a realistic failure to inject and a meaningful one to measure. The change was absorbed entirely by the device registry; no workflow builder was altered, which is itself evidence that the logical-device abstraction achieved what O5 required of it.

4.3 Objectives Not Completed

No objective from Section 2.4 was abandoned. Three capabilities that are discussed in this report are nevertheless *not* delivered, and are recorded here so that they are not read into the contributions above.

- **Ground-truth confirmation of physical actuation.** The prototype contains no sensors. A node's completion reply follows a fixed actuation delay, so an acknowledgment establishes that the node executed its actuation sequence, not that a door moved. This boundary is quantified and its consequences analysed in Sections 7.3 and 7.5, and closing it is the first item of future work.
- **Dependency-aware or parallel scheduling.** Execution is a single sequential loop with a maximum concurrency of one. Ordering encodes the real dependencies — a door is opened before the room beyond it is entered — but that ordering is implicit in the order in which the builder constructs the list rather than declared and enforced. The measured cost of this decision is the 13.8-fold emergency degradation reported in Section 7.2.5.

-
- **An open-loop versus acknowledgement-gated comparison on hardware.** An alternative executor that dispatches without waiting for replies, and the analysis tooling to compare the two, were both implemented and unit-tested, but the comparison was never run against the physical prototype. No such comparison is therefore reported in Section 7, and none is claimed.

5 System Design and Methodology

This section describes the system as implemented. Section 5.1 traces the path a spoken request actually takes; Section 5.2 decomposes that path into architectural layers and describes the environment model that conditions it; Section 5.3 gives the electrical interfacing of the three nodes; and Section 5.4 states the control logic of the eight algorithms that carry the engineering weight of the project.

5.1 Overall Workflow

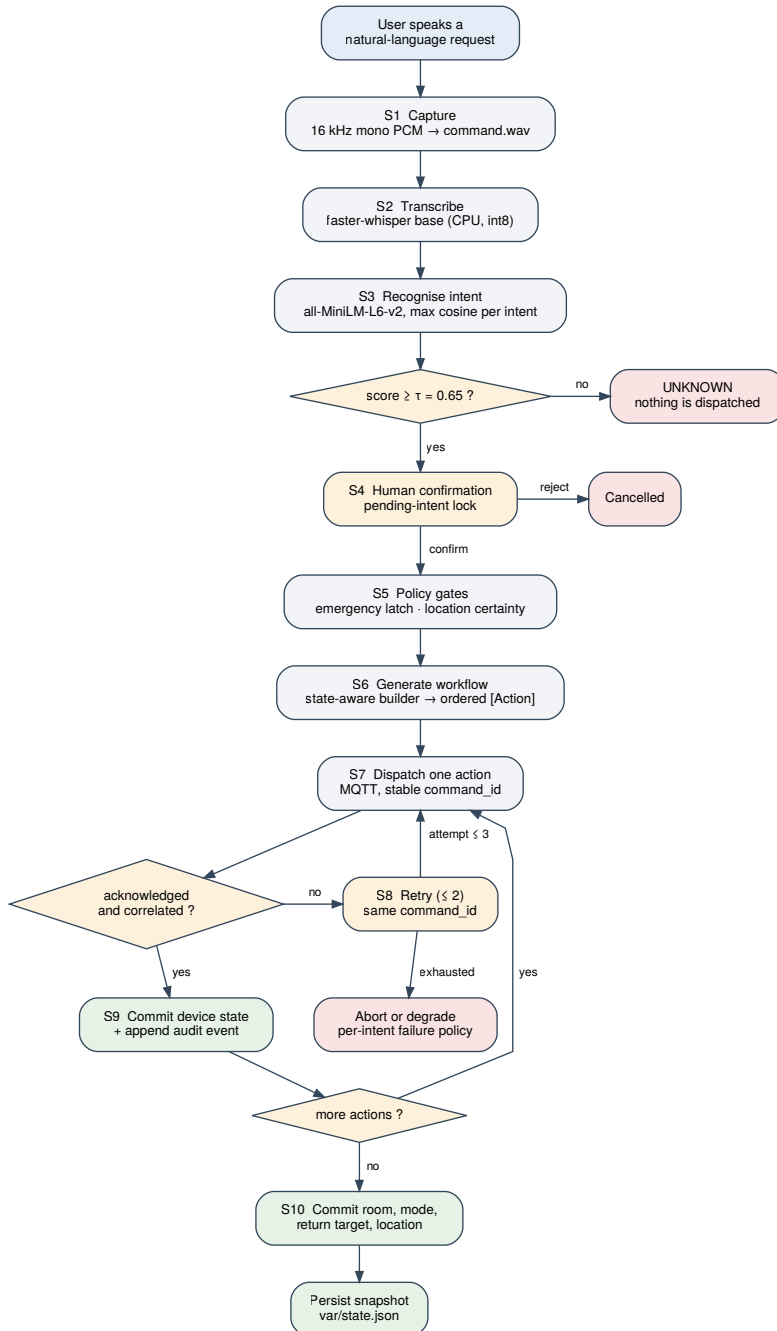


Figure 1. End-to-end workflow of the implemented system. Diamonds are decision points; the two shaded terminal states on the right are the paths on which no hardware is addressed at all. Stages are labelled S1–S10 and are described in the text.

Figure 1 shows the complete path from utterance to persisted state. It represents the delivered implementation rather than an intended design: every decision point shown corresponds to a branch that exists in the code, and the two rejection terminals are reached by measurable fractions of real input, as quantified in Section 7.2.1.

S1 — Capture. Audio is acquired through the host sound device at 16 kHz, single channel, 32-bit float, and written to a WAV file. Recording is explicitly started and stopped rather than triggered by a wake word or by voice activity detection, which removes an entire class of false activations at the cost of requiring a deliberate user action. Concurrent start requests are rejected, an empty capture is reported as such, and a microphone access failure is distinguished from both.

S2 — Transcription. The recording is transcribed by faster-whisper using the `base` model on the CPU with int8 quantisation. Segment texts are concatenated and trimmed. No further preprocessing is applied: there is no voice activity detection, no noise suppression and no loudness normalisation, so transcription quality degrades directly with acoustic conditions. An empty transcript terminates the request.

S3 — Intent recognition. The transcript is embedded and scored against the reference corpus as described in Algorithm 1. The stage emits a decision, the best intent, its similarity score, the nearest reference sentence and the three highest-scoring candidates. The candidate list is displayed to the user for transparency; no control logic branches on it.

S4 — Confirmation. A recognised intent is not executed. It is registered as a pending intent under a lock and presented to the user, who must confirm it explicitly. A confirmation request naming any intent other than the pending one is refused. This is the single most important safety property of the interaction design, because it is the only gate in the system that can catch a confident recognition of an incorrect transcript.

S5 — Policy gates. Two gates are applied before any planning occurs. If an emergency is latched, every intent other than emergency and emergency-clear is refused. If the resident's location is not `KNOWN`, every intent that depends on knowing where they are is refused; the two emergency intents are location-independent by design and pass. A request to clear an emergency when none is active is also refused.

S6 — Workflow generation. The builder for the confirmed intent reads current state and returns an ordered list of actions (Algorithms 2 and 3). If the workflow moves the resident, the location is set to `IN_TRANSIT` with the destination recorded before the first action is dispatched.

S7 — Dispatch. Actions are dispatched strictly one at a time. Each action's device is resolved through the registry to a responsible node, a command identifier is minted once for the logical action, and a JSON command is published at QoS 1 to that node's command topic.

S8 — Acknowledgement and retry. The executor blocks for up to five seconds for a completion reply carrying the same identifier. A timeout causes a retry with the identifier held constant, to a maximum of three attempts. A node that replies with an explicit error is not retried, since that failure is a decision rather than a loss.

S9 — Per-action commitment. The reply is validated against the dispatched action before anything is recorded. Only on success is a single device field mutated and a single line appended to the audit log.

S10 — Workflow commitment. After the loop completes, the resident's room, the operating mode and the return target are committed, the location is confirmed as **KNOWN**, and the whole environment model is written to disk.

5.2 System Architecture and Block Diagram

5.2.1 Layered Software Architecture

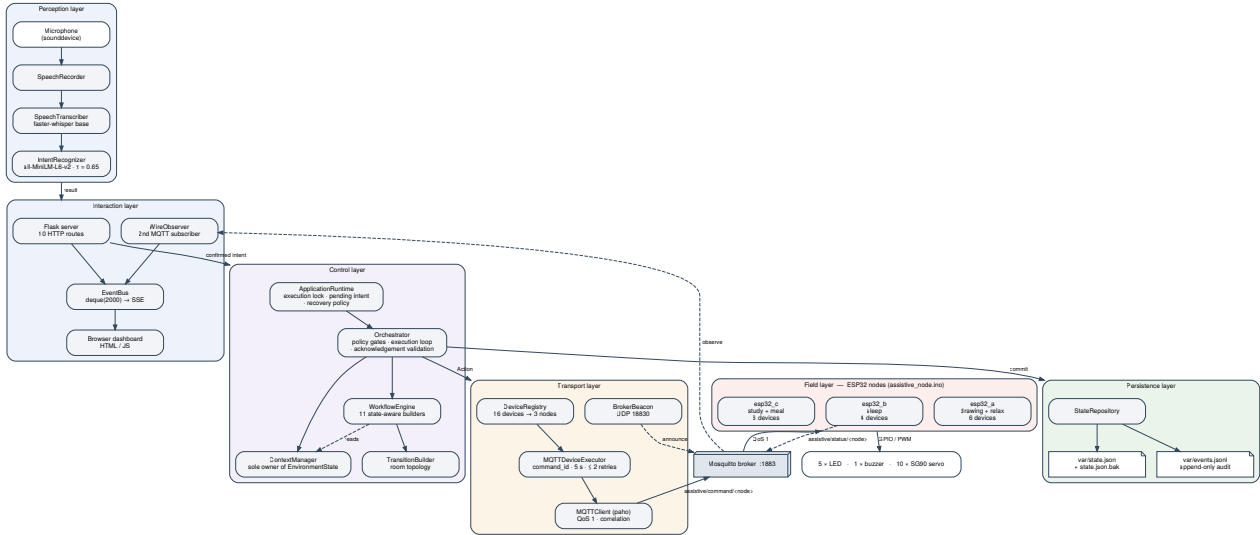


Figure 2. Layered software architecture. Solid arrows carry commands and data in the execution direction; dashed arrows carry replies and observation traffic. Only components that exist and are reachable from an entry point are shown.

Figure 2 shows the six layers and the single direction in which they depend on one another. The dependency chain runs web or console → runtime → orchestrator → workflow engine → transitions → context manager → state, with device executors injected into the runtime rather than imported by it. No circular dependency exists, which is what allows the transport to be replaced wholesale — by a simulated node or by a mock executor — without any change above the registry.

Three features of the diagram deserve comment because they are design decisions rather than consequences of decomposition.

First, **the context manager is the sole owner of environment state.** The workflow engine reads it (shown dashed) but never writes to it; the orchestrator is the only component that calls a mutator, and it does so only after validating a reply. Confining mutation to one component is what makes the commitment rule in Section 5.2.4 enforceable rather than merely conventional.

Second, **the observation path is independent of the execution path.** The wire observer is a second MQTT subscriber with its own client, subscribed to both the command and status topic trees. It timestamps traffic and feeds the event bus that drives the dashboard, but the orchestrator neither knows nor cares that it exists. The consequence is that the dashboard displays genuine bus traffic — including traffic generated by the bench tool rather than by the application — instead of a rendering of what the orchestrator believes it sent. For a project whose central concern is the gap between intended and actual effect, observing the wire rather than the intent is the correct instrument.

Third, **persistence is a consumer of committed actions, not a participant in execution**. The audit log is appended from an observer callback invoked after a commitment has already occurred. An observer that raises an exception is caught and logged and never breaks a workflow.

5.2.2 Hardware Block Diagram and Data/Control Flow

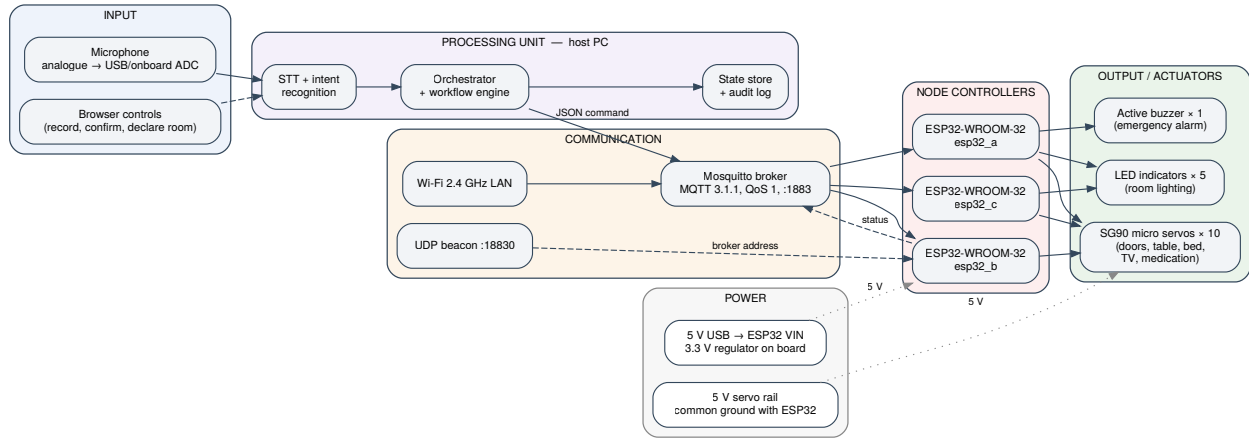


Figure 3. Hardware block diagram showing input devices, the processing unit, the communication path, node controllers, output actuators and the power blocks. Dotted lines are power; dashed lines are replies and discovery traffic. The prototype contains no sensor block, and none is shown.

Figure 3 maps the same system onto physical blocks. The input side comprises the microphone and the browser controls through which the resident starts a recording, confirms an intent, and — when location certainty has been lost — declares which room they are in. All processing of consequence happens on the host: transcription, recognition, planning, verification and persistence. The node controllers are deliberately thin, performing message parsing, duplicate suppression, actuation and reply, and holding no model of the home.

The communication block contains two distinct mechanisms. Command and status traffic flows over MQTT through the broker at QoS 1, with one command topic and one status topic per node. Separately, a UDP beacon broadcasts the broker's address on port 18830 every two seconds, so that a node that cannot reach its broker after repeated failures can rediscover it without being reflashed. This addresses a practical problem of laboratory work — the host receives a different address from the router between sessions — rather than a requirement of the architecture.

The absence of a sensing block is the single most consequential feature of this diagram. Control flows left to right and replies flow right to left, but the reply originates at the node, not at the actuator. The dotted power lines are drawn separately from the signal lines because the servos draw their current from a rail that does not pass through the ESP32.

TO BE CONFIRMED BEFORE SUBMISSION

The power arrangement shown in Figures 3 and 9-11 — ESP32 modules supplied at 5 V through USB with the on-board regulator providing 3.3 V logic, and the ten servos drawing from a separate 5 V rail sharing a common ground with the modules — is the configuration the interfacing diagrams assume. Confirm the actual supply used (USB only, bench supply, or battery), its current rating, and whether decoupling capacitance was fitted across the servo rail, and correct Figures 3 and 9-11 and Table 7 accordingly.

5.2.3 Environment Model and Room Topology

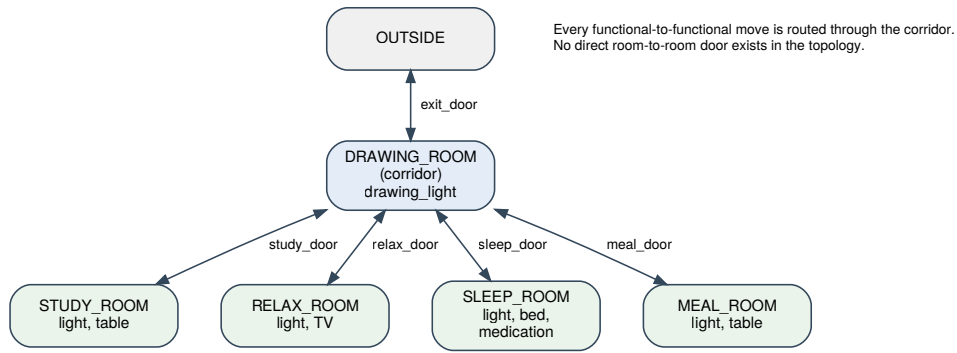


Figure 4. Room topology and door assignment. The drawing room is the sole corridor; no direct door exists between any two functional rooms.

Figure 4 shows the six locations of the environment model. The topology is a star with the drawing room at its centre, and this single structural decision does a disproportionate amount of work. Because every functional-to-functional move must pass through the corridor, a move from the relax room to the sleep room is not a special case to be enumerated but a composition of two known half-moves. Four functional rooms would otherwise require twelve directed room pairs to be specified individually; the star reduces this to one rule about leaving a room and one about entering one.

The transition rules that follow from the topology are given in Table 5. Three invariants hold across all of them. Every door that is opened is closed again in the same sequence, because the resident passes through rather than leaving the door standing open — the emergency workflow is the deliberate exception. The corridor light is state-aware: it is switched on only if it is not already on, and switching it off is the responsibility of the caller, guarded by a predicate that is false when nobody moves, when the corridor is itself the destination, and when the resident is leaving the house. Only the relax room has an appliance switched off on departure, since only it has a television.

Table 3. The eleven supported intents, the actions their builders generate, and the state committed on success. Every builder reads current state and emits only actions that are still required, so the sequences below are maxima rather than fixed lists.

Intent	Generated actions	State committed on success	Failure policy
STUDY_MODE	transition to study room; LIGHT_ON study_light; PREPARE_TABLE study_table; LIGHT_OFF drawing_light if the corridor was used	room = study, mode = STUDY, return target = previous room	fail-fast
RELAX_MODE	transition to relax room; LIGHT_ON relax_light; TV_ON relax_tv; corridor release	room = relax, mode = RELAX, return target	fail-fast
PRE-PARE_FOR_SLEEP	transition to sleep room; LIGHT_ON sleep_light; PREPARE_BED sleep_bed; corridor release	room = sleep, mode = SLEEP, return target	fail-fast
PRE-PARE_FOR_MEAL	transition to meal room; LIGHT_ON meal_light; PREPARE_TABLE meal_table; corridor release	room = meal, mode = MEAL, return target	fail-fast

Intent	Generated actions	State committed on success	Failure policy
MEDICATION	if not already in the sleep room: transition, LIGHT_ON sleep_light, PREPARE_BED sleep_bed, LIGHT_OFF drawing_light; then always ACTIVATE_MEDICATION medication_servo	room = sleep, mode = SLEEP, medication servo = ON	fail-fast
WAKE_UP	RESET_BED sleep_bed, only if currently in the sleep room; otherwise an empty list	mode = NONE, only if previously in the sleep room	fail-fast
RE-TURN_TO_ROOM	transition to the stored return target, that room's setup, conditional corridor release	room = return target, target cleared, mode restored from a room-to-mode map	fail-fast; the builder raises if no return target is stored
LEAVE_ROOM	transition to outside	room = outside, mode = NONE, return target = previous room	fail-fast
SHUT-DOWN_ENVIRONMENT	for each functional room other than the current one, LIGHT_OFF plus any appliance shutdown; for every room, any table or bed reset; then transition to outside. Measured at 11 actions from the drawing room	room = outside, mode = NONE, return target	fail-fast
EMERGENCY	BUZZER_ON if not already on; then OPEN_DOOR for each closed escape door, shared exit first. Closes nothing	mode = EMERGENCY and the latch are set before the first action and are never undone by a failure	best-effort; location-independent
EMERGENCY_CLEAR	BUZZER_OFF if on; CLOSE_DOOR for each open escape door in reverse order, inside doors before the exit	mode = NONE and the latch cleared only if every action was acknowledged	best-effort; location-independent

Table 4. The thirteen-member action vocabulary. An action carries an action type, a logical device identifier and an optional parameter map.

Category	Action types	Devices addressed
Doors	OPEN_DOOR, CLOSE_DOOR	exit_door, study_door, relax_door, sleep_door, meal_door
Lighting	LIGHT_ON, LIGHT_OFF	drawing_light, study_light, relax_light, sleep_light, meal_light
Appliance	TV_ON, TV_OFF	relax_tv
Surfaces	PREPARE_TABLE, RESET_TABLE	study_table, meal_table
Bed	PREPARE_BED, RESET_BED	sleep_bed
Medication	ACTIVATE_MEDICATION	medication_servo
Alarm	BUZZER_ON, BUZZER_OFF	buzzer

Table 5. Room transition rules. Bracketed actions are emitted only if the corresponding device is not already in the required condition. "Dest" and "src" denote the destination and source room respectively.

From → To	Generated action sequence
X → X	empty list — a no-op transition emits nothing
Outside → Drawing room	OPEN exit_door, CLOSE exit_door, [LIGHT_ON drawing_light]
Outside → functional	OPEN/CLOSE exit_door, [LIGHT_ON drawing_light], OPEN/CLOSE dest door
Drawing room → functional	[LIGHT_ON drawing_light], OPEN/CLOSE dest door
Drawing room → Outside	LIGHT_OFF drawing_light, OPEN/CLOSE exit_door
Functional → Drawing room	LIGHT_OFF src light, [TV_OFF if relax], OPEN/CLOSE src door, [LIGHT_ON drawing_light]
Functional → functional	LIGHT_OFF src light, [TV_OFF if relax], OPEN/CLOSE src door, [LIGHT_ON drawing_light], OPEN/CLOSE dest door
Functional → Outside	composition of functional → drawing room followed by drawing room → outside

Table 6. Device registry: the mapping from logical device identifier to responsible node and physical actuator. Workflow builders address only the left-hand column.

Logical device	Node	Room	Actuator	GPIO	Travel
drawing_light	esp32_a	Drawing room	LED	23	on / off
relax_light	esp32_a	Relax room	LED	22	on / off
relax_tv	esp32_a	Relax room	SG90 servo	21	0° → 50°
buzzer	esp32_a	Shared	Active buzzer	19	on / off
exit_door	esp32_a	Shared	SG90 servo	18	90° → 180°
relax_door	esp32_a	Relax room	SG90 servo	5	90° → 180°
sleep_light	esp32_b	Sleep room	LED	23	on / off
sleep_door	esp32_b	Sleep room	SG90 servo	18	90° → 180°
sleep_bed	esp32_b	Sleep room	SG90 servo	17	90° → 135°
medication_servo	esp32_b	Sleep room	SG90 servo	16	90° → 10°, returns
study_light	esp32_c	Study room	LED	23	on / off
meal_light	esp32_c	Meal room	LED	22	on / off
study_door	esp32_c	Study room	SG90 servo	18	90° → 180°
meal_door	esp32_c	Meal room	SG90 servo	5	90° → 180°
study_table	esp32_c	Study room	SG90 servo	17	150° → 90°
meal_table	esp32_c	Meal room	SG90 servo	16	90° → 30°

5.2.4 State Model and the Two-Tier Commit Rule

The environment model holds global fields — current room, location status with its destination and reason, return target, operating mode, emergency declaration and clearance timestamps, and the corridor light, house exit and buzzer — together with a per-room record for each of the four functional rooms. Devices take one of three value types: a door is open or closed, a light, television, buzzer or medication servo is on or off, and a table or bed is ready or normal.

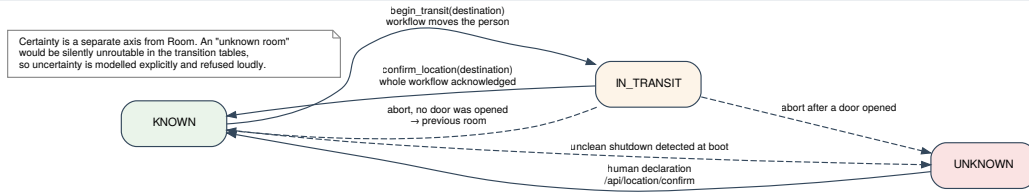


Figure 5. Location-certainty state machine. Solid transitions are the normal path; dashed transitions are loss and recovery of certainty. No transition is triggered by a sensor, because the prototype has none.

Figure 5 shows why location certainty is modelled as a separate axis rather than as an additional room. Rooms are used as dictionary keys in the transition tables and are matched by conditional chains that have no fallback branch, so an "unknown room" value would be silently unroutable — the system would fail to generate a transition and would not be able to say why. Separating certainty from position makes the failure explicit: the orchestrator refuses the request and names the reason, and the resident restores certainty by declaring their room. The abort rules are asymmetric for a physical reason. If a workflow is abandoned before any door has been opened, the resident cannot have moved, so the previous room is reconfirmed. If a door has already been opened, they may have begun to move, so certainty is discarded rather than guessed.

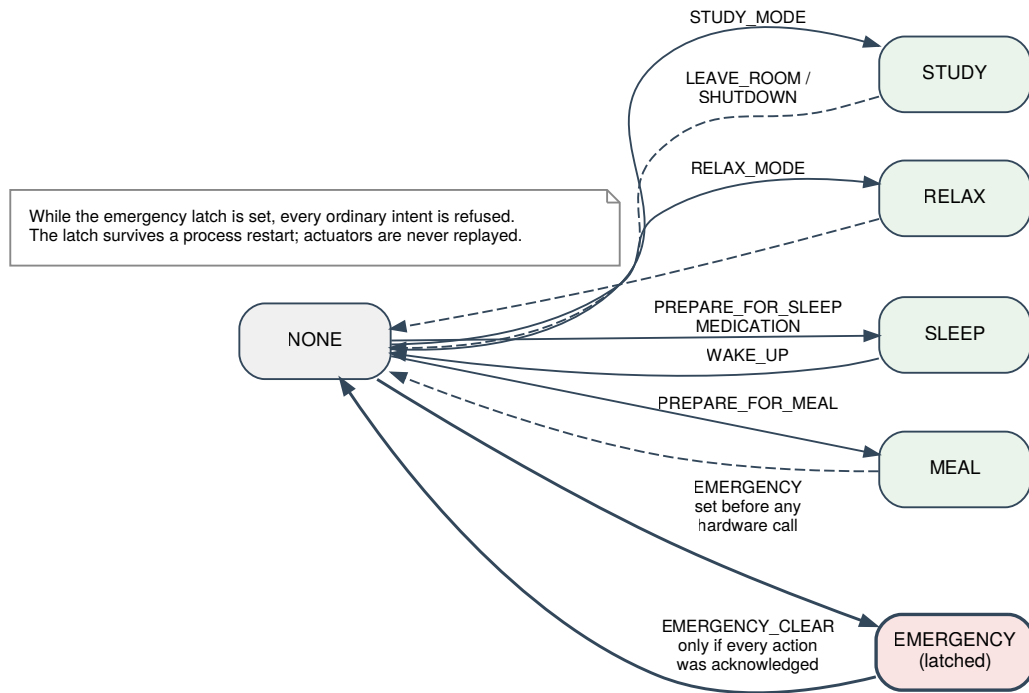


Figure 6. Operating-mode state machine. The emergency state is latched: it is entered before any hardware is addressed, it refuses every ordinary intent while set, and it survives a process restart.

Figure 6 shows the operating modes and the emergency latch. The latch is the one place where the system deliberately commits state *before* acting rather than after, and the asymmetry is intentional: a partly failed alarm is still an emergency, so declaring it early is the conservative error. Clearing is the mirror image and is conservative in the opposite direction — the latch is released only if every action in the clearing sequence was acknowledged, so a partial clear leaves the emergency in force and a repeat request asks only for what is still outstanding.

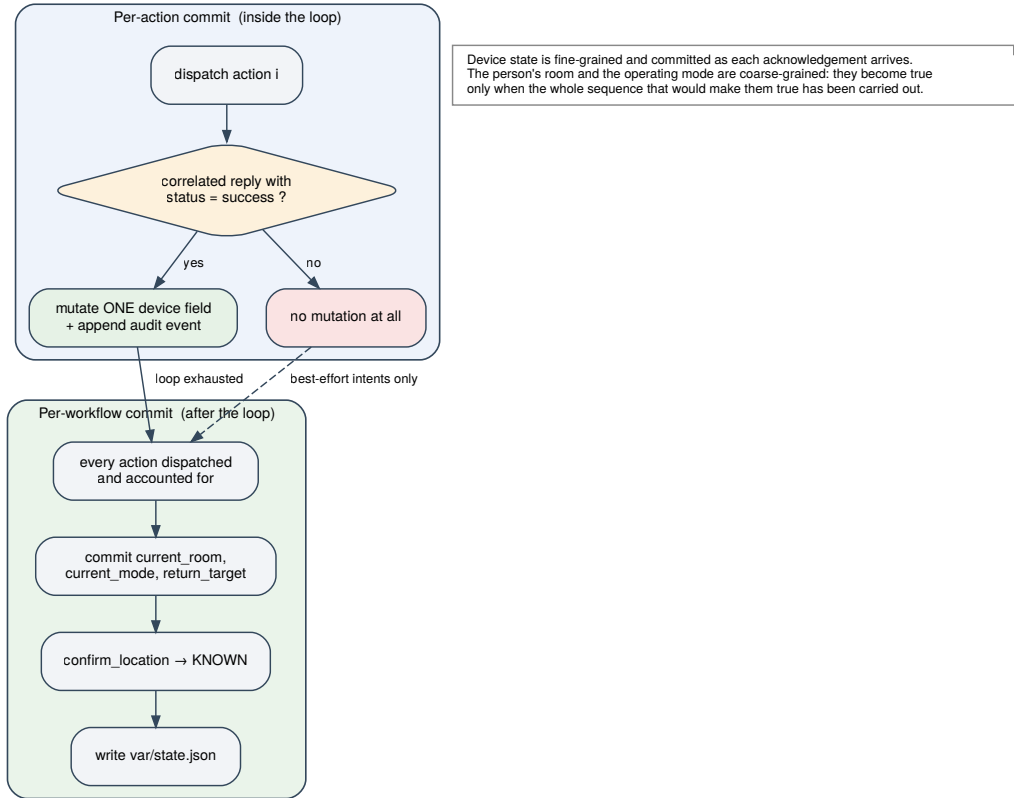
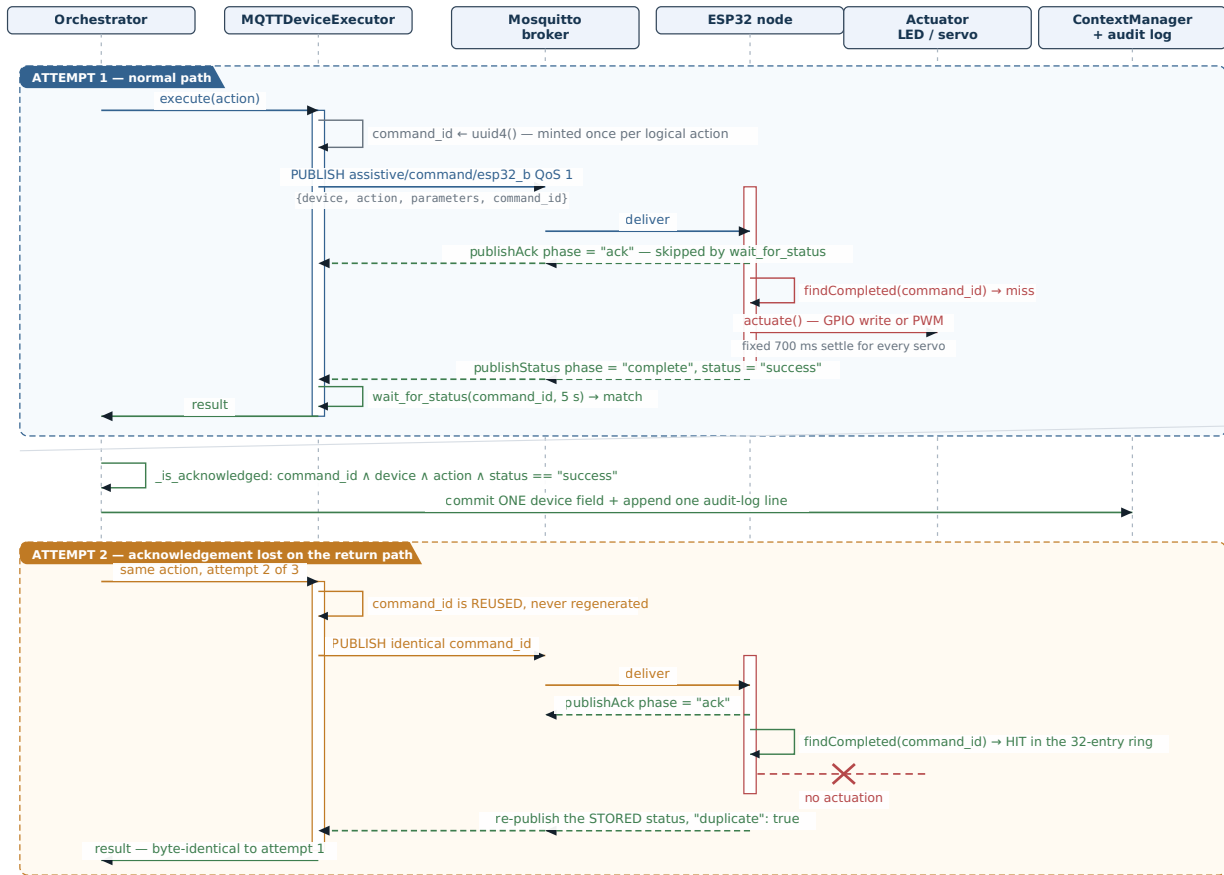


Figure 7. The two-tier state commitment rule. Device fields are committed individually as acknowledgements arrive; the resident's room and the operating mode are committed only after the entire sequence has been executed.

Figure 7 states the rule that distinguishes this system from an optimistic controller. The two tiers exist because the two kinds of fact have different truth conditions. That a particular light is on becomes true the moment that particular node reports it, independently of everything else in the sequence. That the resident is in the sleep room does not become true when the sleep-room door opens; it becomes true only when the whole sequence that would put them there has been carried out. Committing the coarse facts per action would therefore record a resident in a room they had not reached. The practical consequence is that a workflow failing at its fifth action leaves four device fields correctly updated, the room unchanged, and the location marked uncertain — a record that is incomplete but contains no false statement.

5.2.5 Command, Acknowledgement and Retry Sequence



Measured separation on the medication servo: 27 ms for a deduplicated replay against 1 915 ms for a genuine execution — a factor of roughly 72.

Figure 8. Command, acknowledgement and retry sequence. The upper panel is the normal path; the lower panel shows a retry after a lost acknowledgement, in which the reused command identifier causes the node to answer from stored state without actuating.

Figure 8 traces a single action through the transport in both the normal and the retried case, and it is the clearest statement of the project's central mechanism.

Three details in the upper panel matter. The command identifier is minted once per logical action rather than per transmission. The node replies twice — an acknowledgement phase emitted on receipt and a completion phase emitted after actuation — and the executor waits specifically for the completion phase, skipping the acknowledgement. And the completion reply follows a fixed settling delay rather than any confirmation of movement, which is the precise point at which the system's knowledge stops.

The lower panel shows what the stable identifier buys. The retry carries the identifier of the original attempt, so the node finds it in its ring buffer of completed commands, re-publishes the status it stored the first time with a duplicate flag set, and returns without reaching the actuator. The door does not move twice. Because the reply is byte-identical to the original, the orchestrator's commitment is idempotent as well: it writes a field that already holds that value. An earlier design that minted a fresh identifier per attempt made a retry indistinguishable from a new command and was observed to open a door twice; the identifier scheme shown here is what corrected it, and the measurement in Section 7.2.3 quantifies the correction.

5.3 Hardware Interfacing and Circuit Diagrams

Figures 9 to 11 give the interfacing arrangement of each node. One firmware sketch serves all three, specialised at compile time by a node-selection constant, so the three diagrams differ only in their device table and pin assignment.

The interfacing is deliberately simple. Lights are driven directly from a general-purpose output through a series resistor; the buzzer is an active type requiring only a logic level. Servos receive a 50 Hz PWM signal on a single pin and take their supply current from the 5 V rail rather than from the module, because the stall current of ten servos would exceed what the on-board regulator can deliver. The grounds are common, which is required for the PWM signal to be referenced correctly at the servo. No driver integrated circuit is used: the loads are a light-emitting diode, a buzzer and a signal line, none of which requires current amplification.

Two constraints appear in the diagrams and are worth stating explicitly. General-purpose pins 2 and 25 are reserved on the module variant used and are checked at boot rather than assigned. And every servo rest angle is set at or near 90 degrees, which was chosen so that the direction of travel can be reversed in firmware without physically remounting a mechanical arm — a decision that saved repeated disassembly during calibration.

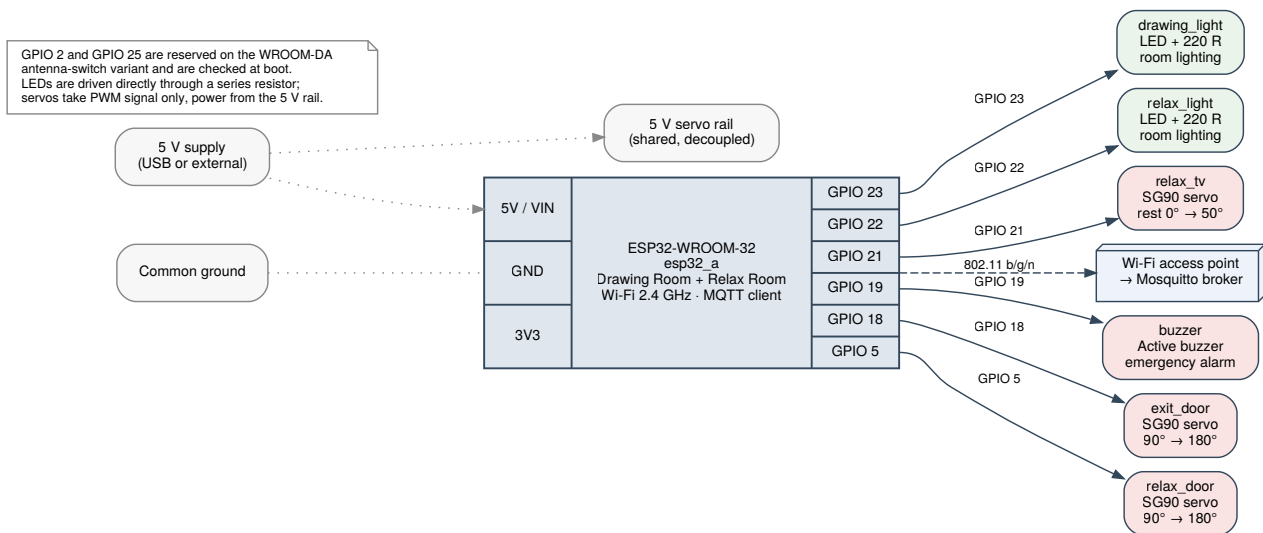


Figure 9. Interfacing diagram for node `esp32_a`, serving the drawing room and relax room together with the house exit and the alarm. Dotted lines are power; the dashed line is the wireless link to the broker.

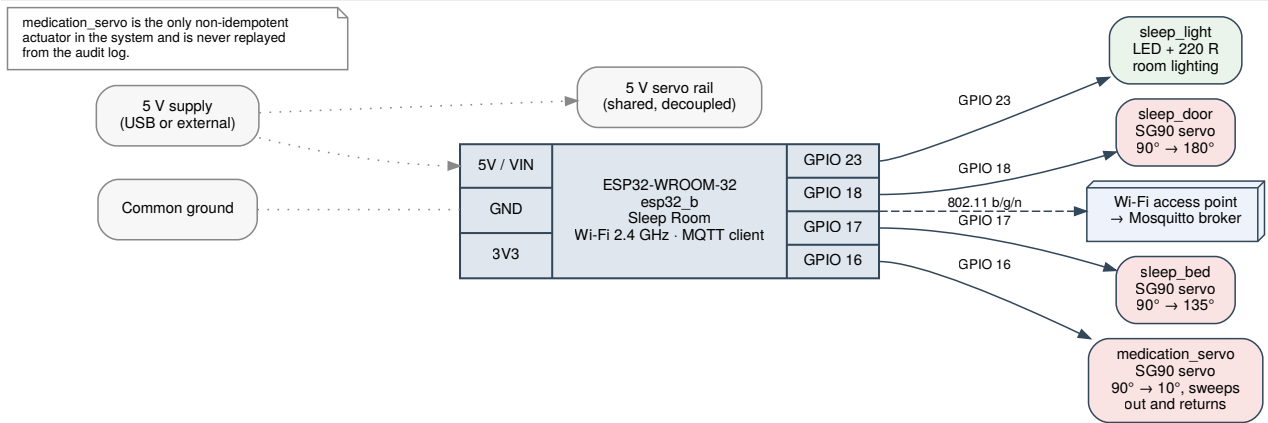


Figure 10. Interfacing diagram for node `esp32_b`, serving the sleep room. The medication servo sweeps out and returns, and is the only non-idempotent physical action in the system.

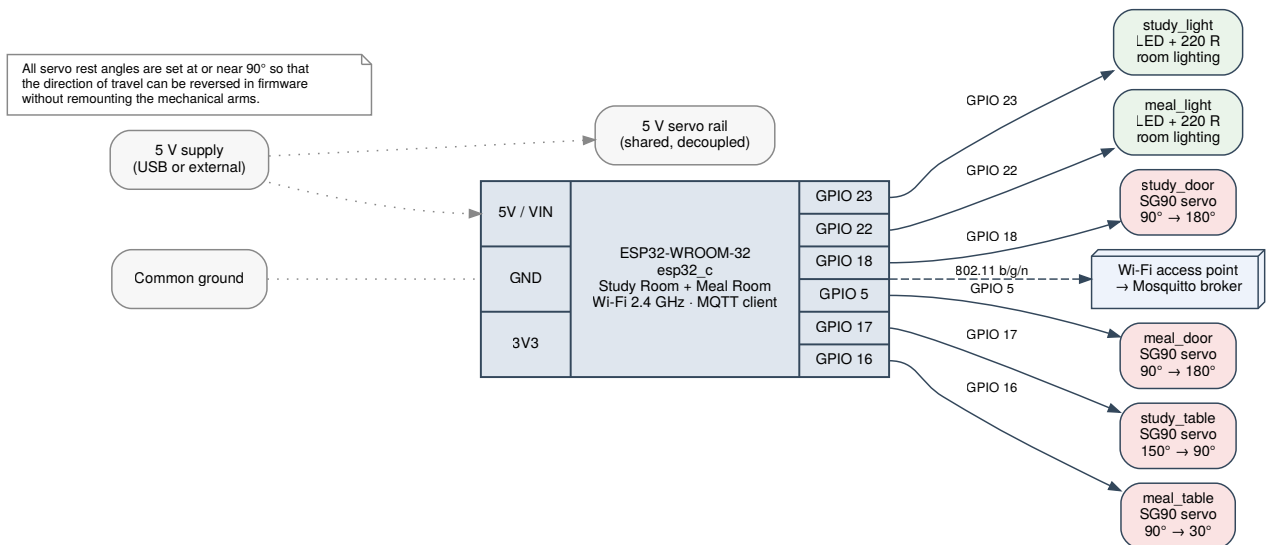


Figure 11. Interfacing diagram for node `esp32_c`, serving the study and meal rooms. This node carries the highest servo count and therefore the highest peak current demand on the shared rail.

5.4 Implementation Logic and Pseudocode

The eight algorithms below express the control and interfacing logic of the system. Routine application code — the browser interface, HTTP route handlers, JSON serialisation and presentation logic — is deliberately omitted.

Algorithm 1 — Semantic intent recognition with explicit rejection

```

Construction (performed once at start-up)
  corpus ← all 313 (sentence, intent) rows of the reference set
  vectors ← encode(corpus.sentences, normalise = true)

predict(transcript, k = 3)
  q ← encode(transcript, normalise = true)
  similarities ← cosine(q, vectors) one score per reference sentence

  an intent scores as its single best sentence, not as a centroid,
  so one well-matched phrasing is sufficient to identify it
  for each intent i in supported_intents
    score[i] ← max( similarities[j] for j where corpus.intent[j] = i )

  best ← argmax(score)
  if score[best] < THRESHOLD (0.65)
    return { decision: UNKNOWN, intent: none, score: score[best],
            top: top_k(score, k) } no workflow is generated
  return { decision: PREDICTED, intent: best, score: score[best],
          matched_sentence: nearest reference sentence,
          top: top_k(score, k) } top_k is displayed, never branched on

```

Algorithm 2 — Room transition generation

```

transition_to(destination)
  current ← state.current_room
  if destination = current
    return [ ] a no-op emits nothing at all

  actions ← [ ]
  if current is a functional room
    actions += LIGHT_OFF room_light[current]
    if current = RELAX_ROOM and relax_tv is ON
      actions += TV_OFF relax_tv only this room has an appliance
    actions += OPEN_DOOR room_door[current]
    actions += CLOSE_DOOR room_door[current] the person passes through
  else if current = OUTSIDE
    actions += OPEN_DOOR exit_door
    actions += CLOSE_DOOR exit_door

  if destination = OUTSIDE
    if drawing_light is ON
      actions += LIGHT_OFF drawing_light
    actions += OPEN_DOOR exit_door
    actions += CLOSE_DOOR exit_door
    return actions

  if drawing_light is OFF
    actions += LIGHT_ON drawing_light corridor lit only if not already lit

  if destination is a functional room
    actions += OPEN_DOOR room_door[destination]
    actions += CLOSE_DOOR room_door[destination]
  return actions

```

Algorithm 3 — State-conditioned workflow generation (room-setup pattern)

```

create_room_workflow(destination, devices_to_prepare)
  previous ← state.current_room
  actions ← transition_to(destination) Algorithm 2

  for each (device, required_action) in devices_to_prepare
    if state_of(device) ≠ effect_of(required_action)
      actions += required_action on device skip what is already true

  if uses_drawing_room(destination)
    false when nobody moves, when the corridor is itself the destination,
    and when the person is leaving the house
    if drawing_light is ON
      actions += LIGHT_OFF drawing_light
  return actions

create_emergency_workflow( )
  actions ← [ ]
  if buzzer is OFF
    actions += BUZZER_ON buzzer alarm first
  for each door in ESCAPE_DOORS shared exit first
    if door is CLOSED
      actions += OPEN_DOOR door
  return actions closes nothing; the route stays open

```

Algorithm 4 — Orchestrated execution with acknowledgement-gated commitment

```

execute_intent(intent)
  if intent ∉ workflow_map raise Unsupported
  if emergency_latched and intent ∉ {EMERGENCY, EMERGENCY_CLEAR}
    raise EmergencyActive
  if intent = EMERGENCY_CLEAR and not emergency_latched
    raise NoEmergency
  if location_status ≠ KNOWN and intent ∉ LOCATION_INDEPENDENT
    raise LocationUnknown

  previous_room ← state.current_room
  previous_mode ← state.current_mode

  if intent = EMERGENCY
    set_mode(EMERGENCY); declare_emergency( ) latched BEFORE any hardware

  actions ← workflow_map[intent]( ) reads current state
  destination ← destination_for(intent)
  if destination ≠ none
    begin_transit(destination) location → IN_TRANSIT
  notify_plan_observer(intent, actions)

  results ← [ ]
  best_effort ← intent ∈ {EMERGENCY, EMERGENCY_CLEAR}

  for each action in actions strictly sequential
    try
      result ← executor.execute(action) Algorithm 5
    catch ExecutionError
      if best_effort
        results += synthetic_failure(action); continue
        abandon_transit(previous_room); re-raise

  if is_acknowledged(result, action)
    correlated by command_id, device and action, and status = success
    apply_effect(action) ONE device field
    notify_action_observer(intent, action, result) audit line
  else if not best_effort

```

```

        abandon_transit(previous_room)
        raise ActionNotAcknowledged
    results += result

    tier two: coarse facts commit only after the whole sequence
    if intent = EMERGENCY_CLEAR
        if all_acknowledged(results)
            set_mode(NONE); clear_emergency( )    partial clear keeps the latch
        else
            update_room_mode_and_return_target(intent, previous_room, previous_mode)
    if destination ≠ none
        confirm_location(destination)    location → KNOWN
    return results

abandon_transit(previous_room)
if no door was opened during this workflow
    confirm_location(previous_room)    the person cannot have moved
else
    mark_location_unknown(reason)    they may have begun to move

```

Algorithm 5 — Device executor with stable identifier and bounded retry

```

execute(action, timeout = 5 s, max_retries = 2)
    node ← DEVICE_NODE_MAP[action.device]
    if node = none    raise UnknownDevice
    command_id ← uuid4( )    minted ONCE for this logical action
    payload ← { device, action, parameters, command_id }

    for attempt in 1 .. max_retries + 1
        publish( "assistive/command/" + node, payload, qos = 1 )
        reply ← wait_for_status(command_id, timeout, phase = "complete")
        the ack phase is ignored; only a completion reply satisfies the wait

        if reply ≠ none and reply.status = "success"
            return reply
        if reply ≠ none and reply.status = "error"
            raise ExecutionError    a decision, not a loss – never retried
            otherwise the wait timed out: retry with the SAME command_id

    raise ExecutionError("no acknowledgement after " + (max_retries+1) + " attempts")

```

Algorithm 6 — Node firmware: message handling with duplicate suppression

```

on_message(topic, payload)
  cmd ← parse_json(payload)
  if parse failed return

  publish_ack(cmd.command_id, phase = "ack")
  sent before the duplicate check, so a replay shows a near-zero ack-to-complete gap

  stored ← find_completed(cmd.command_id) 32-entry ring buffer
  if stored ≠ none
    publish_status(stored, duplicate = true) answer from memory
    return the actuator is NOT reached

  i ← device_index(cmd.device)
  if i = none or not DEVICES[i].wired
    publish_status(error); return

  switch DEVICES[i].kind
    RELAY_LIGHT, RELAY_TV, BUZZER →
      digital_write(DEVICES[i].pin, level_for(cmd.action))
    SERVO_DOOR, SERVO_BED, SERVO_TABLE, SERVO_MED →
      if cmd.action is not valid for this kind
        publish_status(error); return
      servo_write(DEVICES[i].pin, angle_for(cmd.action))
      delay(700) fixed settle; movement is NOT sensed

  remember(cmd.command_id, success) evicts the oldest ring entry
  publish_status(cmd.command_id, status = "success", phase = "complete")

```

Algorithm 7 — Start-up restore and crash-recovery policy

```

restore( )
  snapshot ← read(state.json)
  if unreadable
    snapshot ← read(state.json.bak)
  if still unreadable
    return fresh_state( ) with rejected_reason recorded

  try
    restored ← validate_and_load(snapshot) all-or-nothing
  catch InvalidField
    return fresh_state( ) with rejected_reason recorded

  if snapshot.clean_shutdown = false
    if restored is byte-identical to fresh_state( )
      nothing had happened, so nothing was lost – do not lock out a new user
      return restored
    mark_location_unknown("previous session ended unexpectedly")

  a latched emergency is restored; actuators are NEVER replayed from the audit log,
  which is what keeps ACTIVATE_MEDICATION safe across a restart
  return restored

```

Algorithm 8 — Broker discovery beacon and node re-acquisition

```

host side, every 2 s
    broadcast_udp( port 18830,
        { service: "assistive-broker", broker: local_ip( ), port: 1883 },
        to 255.255.255.255 and to the subnet broadcast address )

node side
    failures ← 0
    loop
        if not mqtt_connected
            if failures ≥ 3
                listen on UDP 18830 until a beacon arrives
                broker_address ← beacon.broker
                failures ← 0
            if connect(broker_address) succeeded
                subscribe("assistive/command/" + NODE_ID)
            else
                failures ← failures + 1
    service_mqtt( )

```

5.5 Diagram Conventions

The same visual conventions are used throughout the report so that the figures can be read against one another. Solid arrows carry commands and data in the direction of execution. Dashed arrows carry replies, acknowledgements and observation traffic. Dotted lines carry power only. Diamonds denote decision points at which the code branches. Shaded green elements denote a committed state change; shaded red elements denote a refusal, an abort or a path on which no hardware is addressed. Bracketed actions in the transition tables are conditional and are emitted only when the target device is not already in the required condition. Every figure and table is numbered, captioned and referred to by number in the surrounding text at the point where it is used.

6 Experimental Setup and Engineering Design Justification

6.1 Components and Equipment

Table 7. Major components of the prototype. Electrical and mechanical figures are typical manufacturer values from the referenced datasheets [18], [19]; measured system-level figures are reported separately in Section 7.2.

Component	Qty	Model / specification	Purpose in the prototype	Important operating range or rating
Node controller	3	ESP32-WROOM-32 module, dual-core Xtensa LX6 to 240 MHz, 520 kB SRAM, 4 MB flash	Receives JSON commands over MQTT, suppresses duplicates, drives actuators, publishes two-phase replies	3.0–3.6 V logic supply via on-board regulator; 2.4 GHz 802.11 b/g/n; recommended continuous GPIO current 20 mA, absolute maximum 40 mA
Micro servo	10	TowerPro SG90, 9 g, plastic gear train, 50 Hz PWM position input	Doors (5), table surfaces (2), bed (1), television (1), medication dispenser (1)	4.8–6.0 V supply; approx. 1.8 kg·cm (0.18 N·m) stall torque at 4.8 V; approx. 0.1 s per 60° travel; usable range approx. 0–180°; stall current of the order of 600–700 mA
Indicator LED	5	5 mm through-hole LED with 220 Ω series resistor	Room lighting state for drawing, study, relax, sleep and meal rooms	Approx. 2.0 V forward drop; approx. 6 mA at 3.3 V through 220 Ω , well inside the recommended continuous pin current
Active buzzer	1	Self-oscillating active buzzer module	Emergency alarm, driven by the emergency workflow	Operates from logic level; requires only an on/off signal, no tone generation
Wireless access point	1	2.4 GHz 802.11 b/g/n router or hotspot	Carries all MQTT traffic and the UDP discovery beacon between host and nodes	Single subnet; broadcast forwarding required for discovery on port 18830
Message broker	1	Eclipse Mosquitto, TCP listener on port 1883, anonymous access	Decouples the executor from the nodes; one command topic and one status topic per node	QoS 1 used throughout; bound to all interfaces on the laboratory subnet
Host computer	1	Windows workstation, Python 3.11.9 virtual environment	Speech capture, transcription, recognition, orchestration, persistence, web server	CPU-only inference; faster-whisper base at int8; Flask bound to the loopback interface
Microphone	1	Host-attached microphone accessed through the sound-device layer	Captures the spoken request	16 kHz sampling, single channel, 32-bit float
Power supply	—	[to be confirmed — see the note below]	Supplies the three modules and the shared servo rail	[rating to be confirmed]

Justification of component selection

Node controller. Three requirements drove this choice: integrated 2.4 GHz wireless networking, because a wired bus to five rooms was not practical; sufficient general-purpose pins to drive six devices from one module after the reserved pins of the module variant are excluded; and enough memory and processing headroom to run a TCP/IP stack, an MQTT

client, a JSON parser and a hardware PWM timer allocator concurrently. The selected module satisfies all three with substantial margin, and its ability to generate hardware PWM on arbitrary pins is what allows the servo assignment in Table 6 to be chosen for wiring convenience rather than being dictated by fixed timer pins.

Servo selection and its limits. The SG90 was selected on cost, availability and the fact that ten units were required. Its stall torque of approximately 0.18 N·m is adequate for the intended load — a lightweight card or acrylic panel representing a door leaf, a hinged table flap, and a small dispensing arm — and its approximately 0.1 s per 60° travel rate is the origin of the 700 ms settling delay adopted in firmware, which allows roughly seven times the nominal traverse time for the largest commanded movement of 90°. This margin is deliberate: because the prototype has no position feedback, the delay is the only mechanism that prevents a completion reply being sent while the arm is still moving. The torque figure is also the clearest statement of what this component cannot do, and Section 7.4 quantifies the gap against a real door leaf.

Rest angle selection. Every servo rest position was set at or near 90°, the centre of usable travel, rather than at an end stop. This permits the direction of travel to be reversed by changing a constant in firmware instead of physically removing and re-seating a horn, and it keeps the commanded excursion within the linear part of the travel range for every device. The travel values in Table 6 were then chosen per device according to the mechanical throw required: 90° for a door leaf, 45° for the bed, 60° for the study table and the tables generally, and a sweep-and-return for the medication arm.

Peak current and the sequential execution policy. Ten servos at an approximate stall current of 650 mA would draw of the order of 6.5 A if commanded simultaneously, which no USB supply and few bench supplies in a teaching laboratory will deliver. The strictly sequential execution loop described in Section 5.1 was adopted for reasons of state consistency, but it has a useful electrical consequence: at most one servo is ever in motion, so the peak demand on the shared rail is bounded by a single device rather than by the population. This is a case where a software decision taken for one reason happens to relieve a hardware constraint, and it should be recognised as such rather than claimed as a power-management design.

Series resistor value. A 220 Ω series resistor with a typical 2.0 V forward drop limits LED current to approximately 6 mA at 3.3 V, roughly one third of the recommended continuous pin current and well below the absolute maximum. The indicators are for state observation during testing rather than for illumination, so brightness was traded for a comfortable margin on the controller.

Transport and quality of service. MQTT was selected over HTTP because the nodes must receive unsolicited commands without being addressable servers, and because a publish-subscribe broker decouples the executor from node addressing entirely. QoS 1 was selected rather than QoS 2: at-least-once delivery is required, since a lost command must be retried, but the four-step handshake of QoS 2 would add latency for a guarantee that — as noted in Section 3.5 — applies only to the client-to-broker hop and still would not establish that the node acted. Duplicate suppression is therefore placed where it can actually protect the actuator, in the node, and QoS 1 is used for the transport.

Table 8. Software stack, with the version used for the results reported in Section 7.

Component	Version	Role
Python	3.11.9	Host application runtime
faster-whisper	1.2.1	Offline transcription, base model, CPU, int8
sentence-transformers	5.7.0	Embedding of transcripts and reference corpus, all-MiniLM-L6-v2
scikit-learn	1.9.0	Cosine similarity; classification metrics in evaluation
pandas	3.0.5	Loading of the intent datasets
sounddevice	0.5.5	Microphone capture
NumPy	2.4.6	Audio buffer handling
Flask	3.1.2	HTTP API and server-sent event stream
paho-mqtt	2.1.0	MQTT client, correlation of replies by command identifier
Mosquitto	external	Message broker
Arduino ESP32 core	3.3.11	Firmware toolchain
PubSubClient	2.8	Firmware MQTT client
ArduinoJson	7.4.3	Firmware JSON parsing and serialisation
ESP32Servo	3.2.1	Firmware PWM timer allocation and servo control
pytest	≥ 8.0	Automated test execution

6.2 Experimental Environment

All measurements reported in Section 7 were obtained under the conditions stated in Table 9. The prototype was not evaluated outside these conditions, and no claim is made about its behaviour beyond them.

Table 9. Experimental environment and the operating envelope within which results were obtained.

Condition	As tested
Deployment setting	Indoor laboratory bench. The five rooms of the environment model are represented by grouped and labelled device clusters, not by physical partitions.
Network	Single 2.4 GHz wireless subnet carrying all MQTT traffic and the UDP discovery beacon. Nodes and broker on the same subnet; broadcast forwarding enabled. [Router or hotspot model and node-to-broker distance to be confirmed.]
Broker	Mosquitto on the host, TCP 1883, anonymous access, no TLS.
Node power	[Supply type and rating to be confirmed — see Section 5.2.2.] Servos drawn from a 5 V rail sharing a common ground with the modules.
Acoustic conditions	Quiet indoor conditions, close-talking microphone, single adult speaker, English. No background-noise, multi-speaker, accent or distance conditions were tested.
Mechanical load	Lightweight panels and flaps representing doors, table surfaces and a bed. No real door leaf, furniture or dispensing mechanism was actuated.
Servo travel	Commanded excursions of 45–90° from a 90° rest position, at room temperature, with no external load opposing travel.
Session length	Bench sessions of the order of tens of minutes. Long-running behaviour — unbounded growth of the reply buffer and of the audit log — was not exercised.
Operator	A single developer-operator. No participant from the target user population took part, and no usability data was collected.

STATEMENT OF TEST-CONDITION LIMITS

The recognition figures in Section 7.2.1 were obtained by passing written sentences directly to the recogniser, not by speaking them. They therefore measure the intent-matching stage in isolation and exclude every transcription error that a real utterance would introduce. End-to-end spoken accuracy would be lower by an amount this project did not measure, and no figure in this report should be read as an end-to-end spoken success rate.

6.3 Measurement Instrumentation and Test Procedure

Four instruments were used, all of them timing on the host with a monotonic high-resolution counter.

Execution measurement. Per-device latency was obtained by dispatching each action individually and recording three instants: the moment of publication, the arrival of the acknowledgement phase, and the arrival of the completion phase. Because the firmware emits its acknowledgement on receipt and its completion after actuation, this decomposition separates transport cost from node-side actuation cost cleanly, and it is the basis of Figures 16 and 17.

Idempotency measurement. The deduplication experiment used a single controlled variable: a firmware compile-time constant that enables or disables the duplicate ring buffer. With the mechanism enabled, each device received an original command followed by four replays of the identical command identifier; with the mechanism compiled out, the same procedure was repeated. Response time and whether the actuator physically moved were recorded for each replay. The build with deduplication disabled prints a warning at start-up and exists solely for this experiment.

Reliability measurement. Three availability conditions were exercised: all nodes reachable; one node powered down for the duration of the trial; and one node powered down and restored part-way through a workflow. Workflow completion, individual task success, retry rate and recovery were counted from observed bus traffic rather than from the orchestrator's own reports, so that a failure to dispatch could be distinguished from a failure to deliver.

Node conformance. A bench tool exercises each node against an eleven-check conformance suite covering device coverage, action validity, reply phase structure, duplicate handling and dropout tracking. It is also the instrument that identified the two divergences between the software simulator and the firmware reported in Section 7.5.

Alongside the hardware instruments, an automated test suite provides deterministic verification of the logic that hardware measurement cannot reach — the commit rule, the emergency lifecycle, the recovery policy, and the transition invariants. Its composition is given in Table 15. The suite requires no broker, no microphone, no model download and no hardware, which is what allows it to be a regression instrument rather than an experiment.

7 Results and Analysis

7.1 Final Prototype and Functional Results

Figure 12 — photographic record of the final prototype

Insert photographs here, ideally as a four-panel plate:

- (a) the complete bench with all three nodes and the five labelled room clusters;
- (b) node `esp32_b` with the sleep-room servos and the medication arm;
- (c) a door mechanism at rest at 90° and at the commanded 180° position;
- (d) the emergency condition, with escape doors open and the buzzer active.

Figure 12. Final prototype. Each panel must be referred to individually in the text once the photographs are inserted.

Figure 13 — browser dashboard during a workflow

Insert a screenshot showing, in one view: the transcript, the recognised intent with its similarity score against the 0.65 threshold, the confirmation control, the generated plan, and the live wire trace of commands and status replies as the workflow executes.

Figure 13. Browser dashboard during execution of a workflow, showing recognition output, the confirmation gate and live bus traffic.

ACTION REQUIRED BEFORE SUBMISSION

Section 7.1 of the assessment guidelines requires photographs of the final prototype, the major hardware setup, important operating stages, inputs with their corresponding outputs, and the user interface. Figures 12 and 13 are placeholders. Insert the images and add one or two sentences of discussion per panel; the quantitative results in Sections 7.2 onward are complete and require no further work.

Functionally, the delivered prototype performs the complete path of Figure 1. A spoken request is transcribed locally, matched to one of eleven intents, presented for confirmation, expanded into a state-dependent action sequence, dispatched action by action to three ESP32 nodes, committed only where the responsible node acknowledged the action, and persisted to disk. The following behaviours were demonstrated repeatedly during bench operation and each is quantified in the subsections that follow: expansion of a single intent into an eleven-action multi-node sequence; refusal of an out-of-scope request without any hardware being addressed; refusal of every ordinary intent while an emergency is latched; re-

fusal of location-dependent intents after an unclean restart until the resident declares a room; suppression of a replayed command at the node; and restoration of the full environment model across a process restart.

7.2 Performance Evaluation

7.2.1 Intent Recognition Accuracy

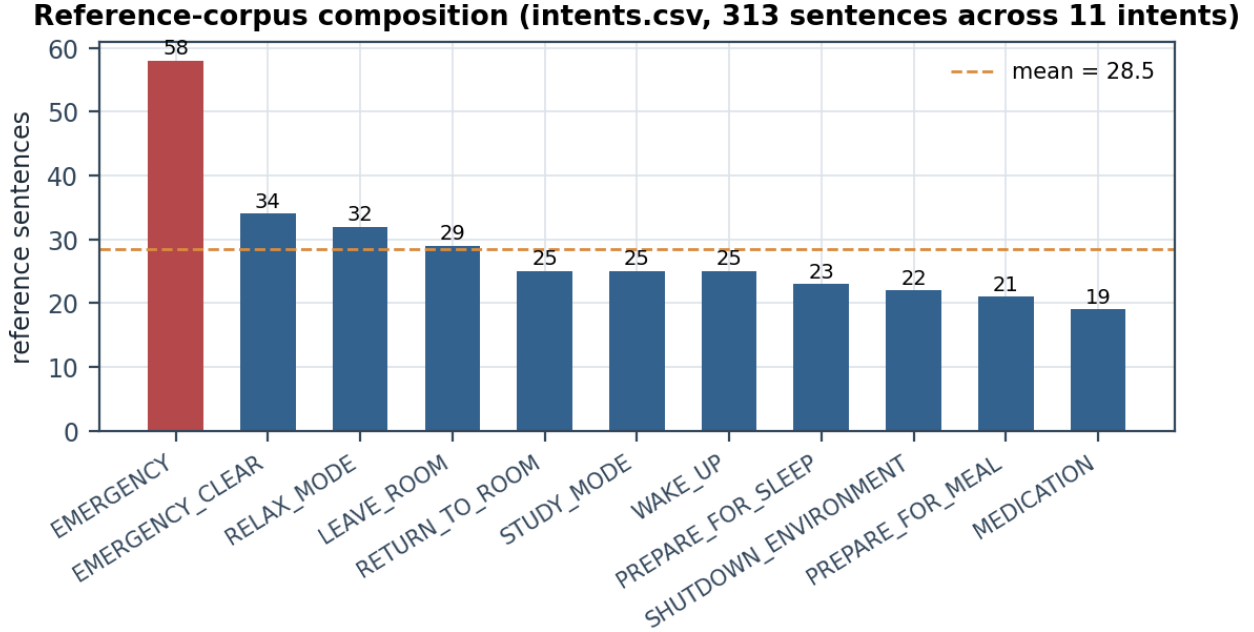


Figure 14. Composition of the reference corpus. Distribution across the eleven intents is uneven, with the emergency intent holding roughly three times the representation of the medication intent.

Figure 14 shows the reference corpus that the recogniser matches against: 313 sentences across eleven intents, ranging from 58 sentences for the emergency intent down to 19 for medication, against a mean of 28.5. Nothing is trained on this corpus — the encoder is frozen and the corpus is a retrieval set — so the distribution does not bias a decision boundary in the way an unbalanced training set would. It does, however, determine how many distinct phrasings of each activity the system can recognise, and the intents at the right-hand end of the figure are correspondingly more fragile to unfamiliar wording.

Evaluation used two sets disjoint from the reference corpus: a balanced holdout set of 110 sentences, exactly ten per intent, and a validation set of 105 sentences ranging from eight to ten per intent. Exact overlap with the reference corpus is zero in both cases. Near-duplicate paraphrase remains: at a Jaccard similarity of 0.8 or above, seven validation sentences and four holdout sentences resemble a reference sentence, which is acknowledged as leakage in Section 7.5 and is the likely source of part of the gap between the two sets.

Table 10. Intent recognition results on two disjoint evaluation sets. Rejection is the fraction scoring below the 0.65 threshold and returning `UNKNOWN`. The substitution column is derived by subtraction and denotes utterances assigned to the wrong intent.

Evaluation set	n	Accuracy	Rejection	Substitution	Macro F1	Weighted precision
Holdout (balanced, 10 per intent)	110	0.718	0.282	0.000	0.74	1.00
Validation	105	0.752	0.162	≈ 0.086	0.75	0.93

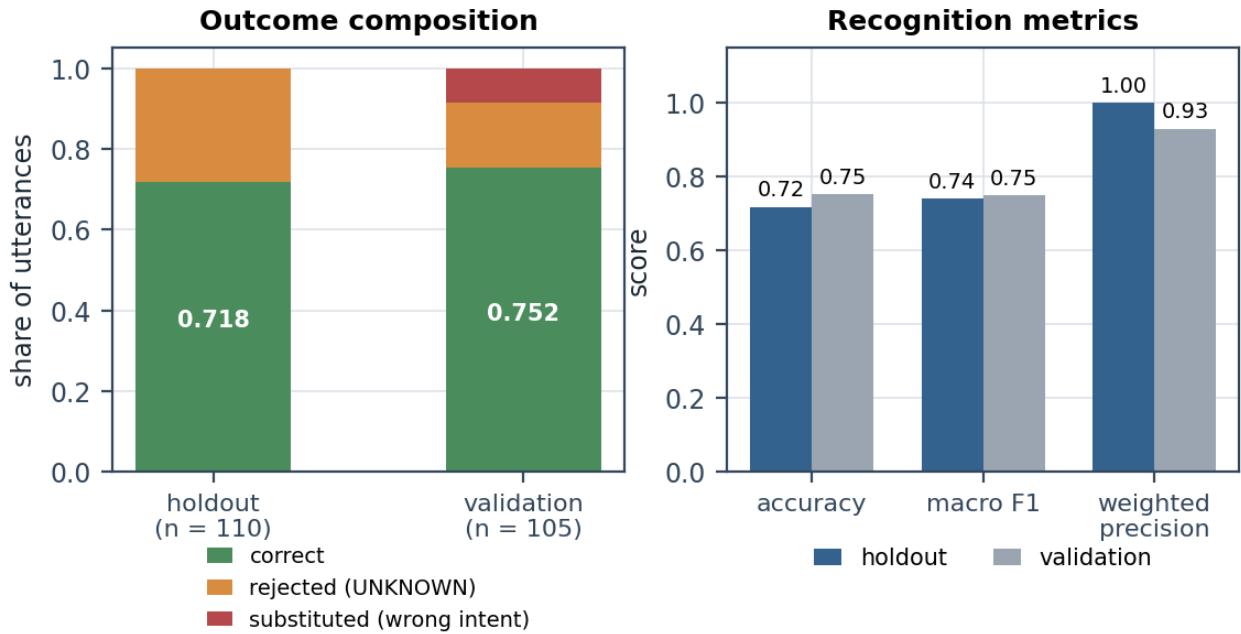


Figure 15. Intent recognition outcomes and metrics. On the holdout set the error bar is composed entirely of refusals; on the validation set a small substitution component appears.

The headline accuracy figures — 0.718 on holdout and 0.752 on validation — are modest and are not the interesting part of Table 10. The interesting part is the composition of the errors. On the holdout set, precision is 1.00 for every one of the eleven intents. All thirty-one errors are refusals; not one is a substitution of one activity for another. In behavioural terms the system never once opened the wrong door: it either did what was asked or declined and asked the resident to repeat. On the validation set the picture is slightly worse, with a weighted precision of 0.93 implying roughly nine substitutions in 105 utterances, which is why the holdout set — balanced, and the stricter of the two — is treated as the primary result throughout this report.

This error profile is a direct consequence of the design choices in Section 5.4. The per-intent score is a maximum over reference sentences rather than a centroid, so an intent wins only when some individual phrasing genuinely resembles the input; a global threshold then refuses everything that resembles nothing strongly. The resulting system trades recall for precision, and for an interface whose outputs are physical doors that is the correct direction of the trade. An accuracy of 0.718 means roughly one request in four must be repeated, which is a usability cost; a substitution rate of zero means no request is ever silently converted into a different activity, which is a safety property.

7.2.2 Execution Latency

Table 11. Measured per-device execution latency on the physical prototype. Dispatch-to-ack is transport and node parse time; ack-to-complete is the node's actuation sequence.

Device	Node	Dispatch → ack	Ack → complete	Total
sleep_light	esp32_b	17 ms	69 ms	86 ms
drawing_light	esp32_a	17 ms	56 ms	73 ms
relax_light	esp32_a	22 ms	59 ms	81 ms
buzzer	esp32_a	19 ms	58 ms	77 ms
sleep_door	esp32_b	18 ms	702 ms	720 ms
sleep_bed	esp32_b	16 ms	704 ms	720 ms
relax_door	esp32_a	18 ms	701 ms	720 ms
relax_tv	esp32_a	21 ms	704 ms	724 ms
exit_door	esp32_a	30 ms	691 ms	721 ms
medication_servo	esp32_b	13 ms	1 902 ms	1 915 ms

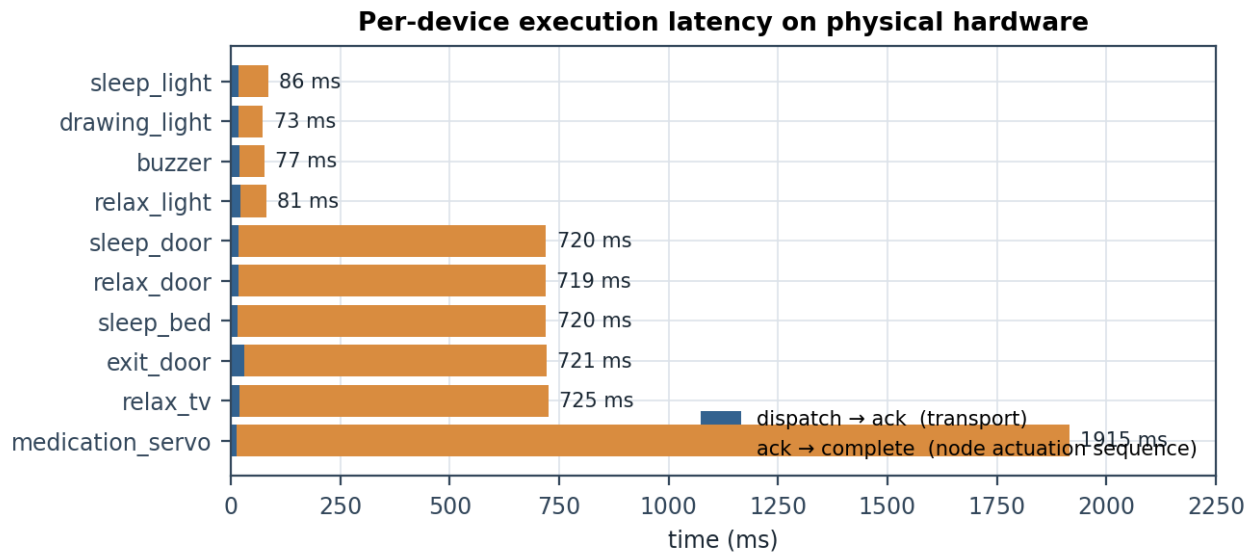


Figure 16. Per-device execution latency on physical hardware. Transport cost is effectively constant across devices; the variation is entirely in the node's actuation sequence.

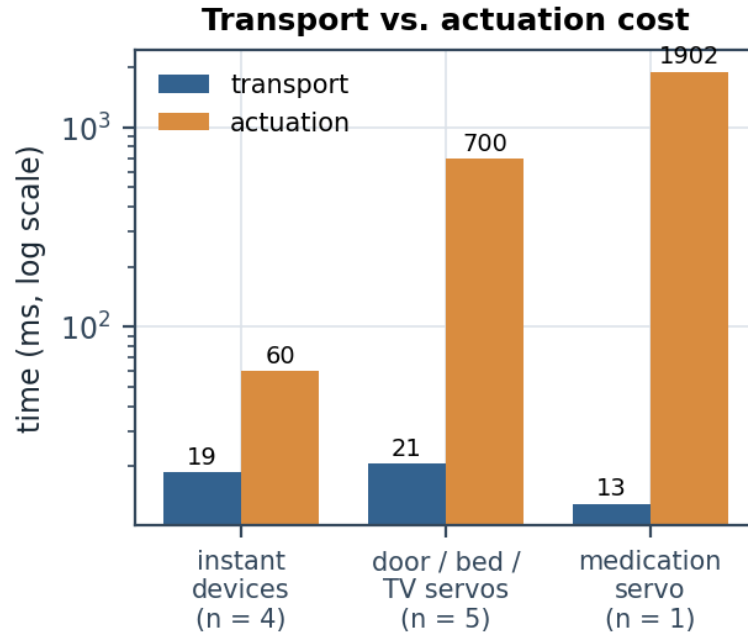


Figure 17. Transport cost against actuation cost, grouped by device class, on a logarithmic scale. Transport varies by a factor of about two; actuation varies by a factor of about thirty.

Figures 16 and 17 show that the transport is not the bottleneck and, for the purposes of this project, is close to irrelevant. Dispatch-to-acknowledgement ranges from 13 to 30 ms across every device and every node — a spread of 17 ms that includes broker round-trip, wireless transmission, JSON parsing on the node and the reply publication. Against a five-second acknowledgement window this consumes at most 0.6 per cent of the budget, which is why a timeout of five seconds is not merely adequate but roughly two orders of magnitude more generous than the observed distribution requires.

Everything else is actuation. Instant devices complete in 56 to 69 ms; every servo except one completes in 691 to 704 ms, tightly clustered around the fixed 700 ms settling delay written into the firmware. The clustering is itself informative: the measured spread of 13 ms across five mechanically different servos confirms that the figure being measured is the programmed delay and not the mechanical travel. The medication device is the outlier at 1 902 ms because it performs two movements — a sweep out and a return — and therefore incurs the delay twice plus the intervening motion.

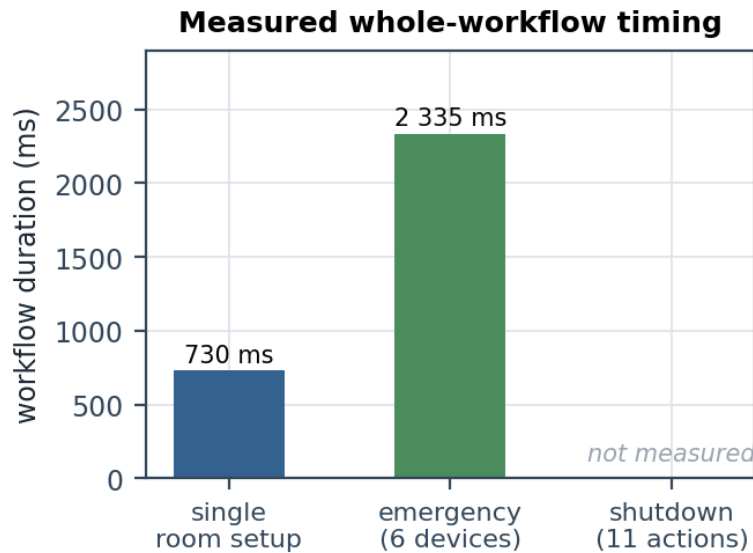


Figure 18. Measured whole-workflow timing. The eleven-action shutdown sequence was generated and verified but was not separately timed on hardware.

At the workflow level, a single-room setup sequence completes in approximately 730 ms and a healthy six-device emergency sequence in a median of 2 335 ms. Both are consistent with the per-device figures accumulated sequentially, which is the expected result given that the executor never overlaps actions.

7.2.3 Idempotency Under Retry

This is the project's principal experimental result. The experiment isolates a single controlled variable — the firmware constant that enables node-side duplicate suppression — and holds everything else fixed.

Table 12. Replay response time and physical re-execution count, with duplicate suppression enabled and with the same mechanism compiled out. Four replays of an identical command identifier were issued per device per condition.

Device	Replay, dedup on	Replay, dedup off	Re-executed, on	Re-executed, off
sleep_door	33 ms	717 ms	0 / 4	4 / 4
sleep_bed	12 ms	714 ms	0 / 4	4 / 4
medication_servo	27 ms	1 915 ms	0 / 4	4 / 4
sleep_light	25 ms	14 ms	not separable	not separable

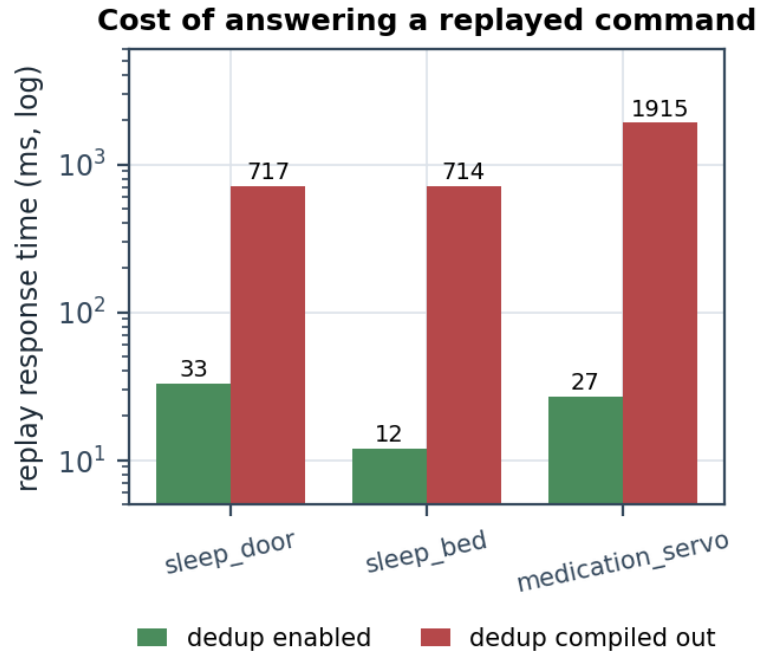


Figure 19. Response time for a replayed command, logarithmic scale. With suppression enabled the reply is served from stored state; with it removed the device actuates again and the reply costs a full actuation cycle.

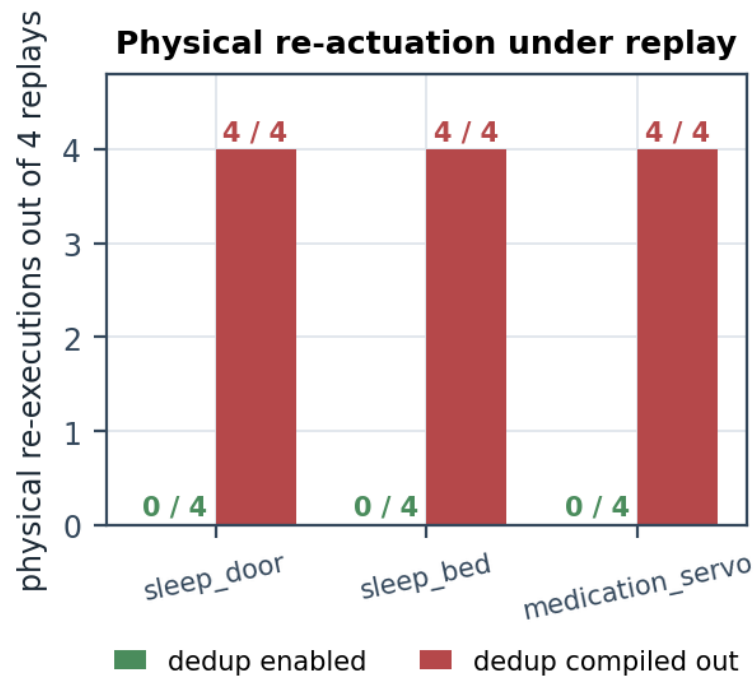


Figure 20. Physical re-actuation under replay. The separation is total: every replay is suppressed with the mechanism enabled, and every replay re-actuates without it.

Across all ten instrumented devices, forty replays out of forty were answered from stored node state with zero physical re-executions while suppression was enabled. With the mechanism compiled out, every servo tested re-executed on four replays out of four. The result is categorical rather than statistical: the mechanism either intercepts the duplicate before the actuator or it does not, and the experiment confirms that it does.

A second, unplanned result follows from the firmware's reply ordering. Because a node acknowledges on receipt but completes only after its actuation sequence, a replay answered from stored state costs 27 ms on the medication device against 1 915 ms for a genuine execution — a separation of roughly seventy-two times. Re-execution is therefore detectable from timing alone on slow actuators, without any sensor. The limit of this observation is stated rather than hidden: on the instant devices the separation collapses, with a suppressed replay of the sleep light measured at 25 ms against 14 ms for a real switch operation. The timing signal cannot distinguish the two cases where actuation is fast, so it is a useful diagnostic for servos and useless for lights.

7.2.4 Reliability, Containment and Recovery

Table 13. Reliability under three node-availability conditions. Workflow completion counts sequences that ran to the end; task success counts individual actions that were acknowledged; recovery counts workflows that completed after a node returned.

Condition	Workflow completion	Task success	Retry rate	Recovery
All nodes reachable	12 / 12	60 / 60	0	not applicable
One node unreachable throughout	9 / 12	21 / 30	0.60	0 / 9
Node returns part-way through a workflow	12 / 12	60 / 60	0.18	6 / 6

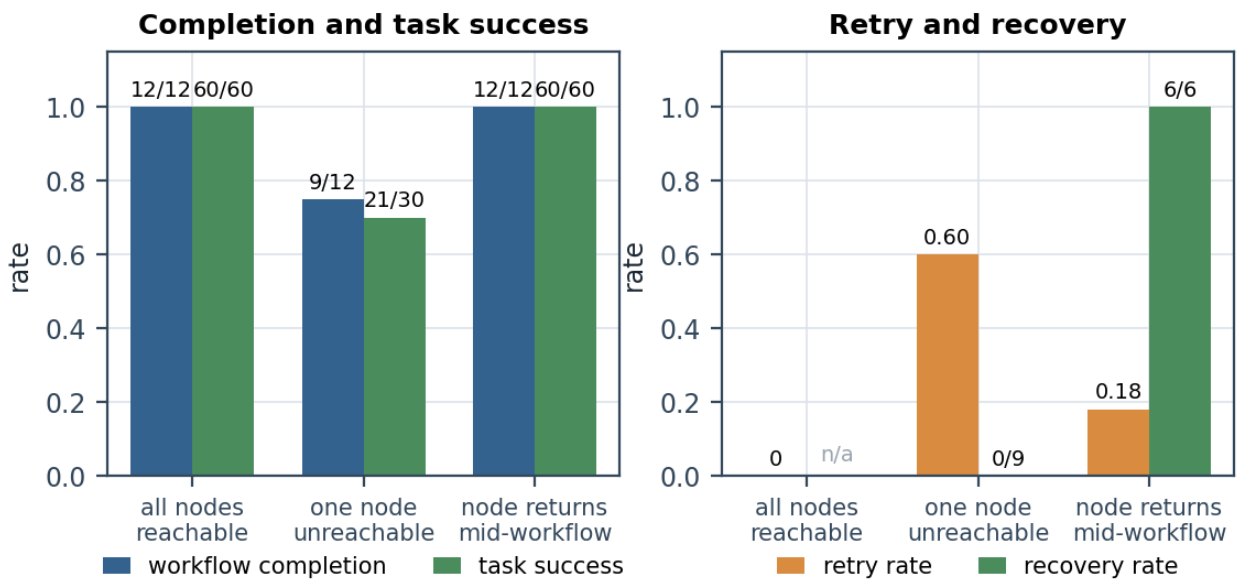


Figure 21. Reliability, retry and recovery across the three availability conditions. Transient loss is recovered completely; sustained loss is not recovered at all, which is the intended behaviour of a bounded retry budget.

Three findings emerge from Table 13. With every node reachable the system is clean: sixty actions out of sixty succeeded across twelve workflows with no retry at all, confirming that the retry mechanism is not masking routine transport loss on this network. When a node is permanently unreachable, nine workflows out of twelve still complete — those that never address the missing node — while the retry rate rises to 0.60 and recovery is zero out of nine, which is correct rather than disappointing: a bounded retry budget is meant to give

up on a device that is genuinely absent, and a system that recovered here would be one that retried indefinitely.

The third condition is where the design pays off. When a node is lost and then restored part-way through a workflow, all six affected workflows recovered and every one of sixty actions eventually succeeded, at a retry rate of 0.18. This is precisely the failure the retry budget exists to absorb — a transient dropout shorter than three attempts — and the mechanism absorbed all of it.

Failure containment under the fail-fast policy was exact. After an abort, none of the thirty remaining planned actions across the trial set was dispatched, giving a containment rate of 1.0. This is the property that makes the two-tier commit rule meaningful: not only are uncommitted facts left uncommitted, but the actions that would have justified them are never attempted, so the physical environment does not drift further from the recorded model after the decision to abort has been taken.

7.2.5 Degradation of Best-Effort Execution

Table 14. Emergency workflow duration with all escape devices reachable and with two of six unreachable. Best-effort execution continues past failures rather than aborting.

Condition	Median	Minimum	Maximum
All 6 escape devices reachable	2 335 ms	2 211 ms	2 712 ms
2 of 6 unreachable	32 256 ms	32 237 ms	32 435 ms

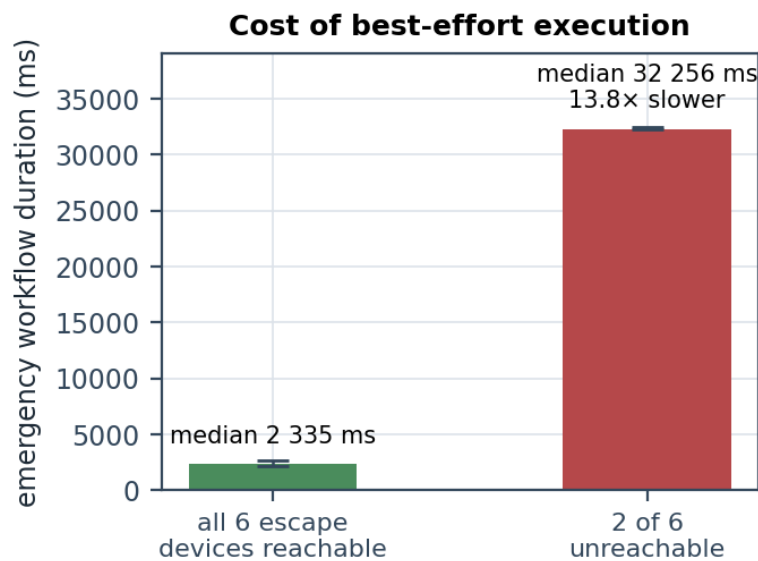


Figure 22. Degradation of best-effort emergency execution. Bars show the median; whiskers span the measured minimum and maximum across trials.

This is a negative result about the system's own design and is reported as such. The emergency workflow uses a best-effort failure policy: rather than aborting when a device cannot be reached, it records the failure and proceeds to the next action, so that every reachable escape route is still opened. The policy is correct for an evacuation — an alarm that stops because one door is unreachable is worse than one that opens the remaining four — but combined with strictly sequential execution it has a severe cost. Losing two devices out of

six increases the median duration from 2 335 ms to 32 256 ms, a degradation factor of 13.8.

The magnitude is fully explained by the retry budget. Each unreachable device consumes three attempts of five seconds, so two unreachable devices contribute $2 \times 3 \times 5 \text{ s} = 30\,000 \text{ ms}$ of pure waiting. The four reachable devices contribute approximately four sixths of the healthy 2 335 ms, or about 1 557 ms. The predicted total of roughly 31 557 ms sits within about two per cent of the measured median of 32 256 ms, and the very narrow observed spread of under 200 ms confirms that the duration is dominated by fixed timeouts rather than by variable network behaviour. The measurement is therefore not merely an observation but a validated model of the system's worst-case behaviour, and it identifies exactly which two constants — the five-second window and the three-attempt budget — would have to be made adaptive to fix it.

7.2.6 Software Verification

Logic that hardware measurement cannot reach was verified by a deterministic automated suite. Of 515 collected tests, 460 run by default and 55 carry a marker excluding them because they load the sentence-embedding model. The last recorded full run produced 458 passed, 2 skipped and 55 deselected; the two skips are deliberate, arising from a dataset check that tolerates the evaluation sets being smaller per label than the reference corpus. The default suite requires no broker, no microphone, no hardware and no model download, and redirects its state directory per test, which is what makes it deterministic.

Table 15. Composition of the automated test suite, grouped by the property under test. Model-marked suites are excluded from a default run.

Area under test	Tests	Property verified
Start-up and recovery	36	Restore policy, clean and unclean shutdown, backup use, survival of the emergency latch
Emergency lifecycle	32	Latch, refusal of ordinary intents, clear, partial clear, best-effort behaviour
State persistence	26	Atomic write, backup file, schema version, strict all-or-nothing restore
Location semantics	23	The three certainty values and the asymmetric abandon-transit rules
Web confirmation flow	20	Pending intent, confirm and cancel, blocked intents, degraded outcomes
Emergency recognition (model-marked)	19	Recall on emergency phrasings
Continuous runtime	15	Console loop; state preserved between successive commands
MQTT end to end	14	Executor against a simulated node over a fake transport
Transition invariants	14	Corridor routing rules and door open/close pairing
State-commit regressions	13	Acknowledgement-gated commitment; the two-tier boundary
Command idempotency	12	Identifier stability across retries; duplicate suppression; no double actuation
Dataset hygiene	12	Dataset shape and per-label counts

Area under test	Tests	Property verified
Web application start-up	12	Application boots; settings match the controller signature
Broker discovery	9	Beacon payload over a real loopback socket; tolerance of errors
Intent contract (model-marked)	8	Decision vocabulary; every predicted intent is a supported intent
Baseline analysis	8	Ordering-violation detection on synthetic event lists
Executor parity	5	Mock and MQTT executors reach identical state; no script bypasses the orchestrator
Workflow protocol, firmware, coverage, workflow map	12	Action JSON encoding; static checks on the firmware sketch; every action type is handled by some node; workflow-map integrity

7.3 Analysis of Results

What the results establish

The orchestration concept works and the expansion factor is real. A single intent produces up to eleven actions spanning three nodes, and the sequence differs run to run because it is computed from state. This is the difference between the system and a command mapper, and it is visible in the measurements rather than only in the design.

Acknowledgement gating costs almost nothing. Transport occupies 13 to 30 ms against actuation costs of 56 to 1 902 ms. Waiting for a correlated reply therefore adds no meaningful latency to a system whose time is spent moving servos. The common objection to closed-loop command execution — that the round trip is too expensive — is not supported at this scale.

Identifier-stable retry eliminates duplicate actuation completely. The controlled comparison is unambiguous: 0 re-executions in 40 replays with the mechanism, 4 of 4 per servo without it. Because the two builds differ in exactly one compile-time constant, the effect is attributable to the mechanism and not to any other property of the system.

Rejection-dominant recognition is achievable without fine-tuning. A frozen encoder, a maximum-over-sentences scoring rule and a single threshold produced a weighted precision of 1.00 across eleven intents on a disjoint balanced set. No intent was ever substituted for another.

The retry budget absorbs exactly the failures it is designed for. Transient loss was recovered in six of six cases; sustained loss was recovered in none, by design.

Where the system performs poorly, and why

Recognition recall. Roughly one holdout utterance in four is refused. The cause is structural: 313 reference sentences across eleven activities cannot cover the space of natural phrasings, and with a frozen encoder the only available lever is the threshold. Lowering it would convert refusals into acceptances, but since the current substitution rate on holdout is zero, that trade would be paid for in exactly the currency the design is trying to protect. The literature indicates the correct fix — Zhang *et al.* [8] show a 1 to 4 percentage point gain in rejection performance from fine-tuning the encoder with an appropriate regulariser — and that is a change to the model rather than to the threshold.

Emergency latency under partial failure. The 13.8-fold degradation analysed in Section 7.2.5 is the worst quantitative result in this report, and the situation in which it occurs — an evacuation with some devices unreachable — is the one in which delay matters most. The arithmetic shows the cause to be entirely the interaction of two design decisions, sequential execution and a fixed retry budget, neither of which is individually unreasonable.

Sustained node loss produces a silent regional failure. When one node is down, nine workflows out of twelve complete normally because they do not address that node. The resident receives no indication that a third of the home has become uncontrollable until they ask for something in it. There is no heartbeat, only an on-demand probe.

The boundary of what the acknowledgement proves

CENTRAL LIMITATION, STATED PRECISELY

A completion reply establishes that the node received a correlated command, matched the device and the action, executed its actuation routine, and waited out its settling delay. It does not establish that anything moved. The firmware waits a fixed 700 ms whether or not a servo is attached or powered, and during data collection for this project six servos lost power and all six reported success, after which the system committed the corresponding device states. The prototype's verification therefore closes the loop at the node, not at the actuator, and every reliability figure in Section 7.2 must be read as a measurement of that narrower loop.

This boundary deserves proportionate treatment rather than either concealment or overstatement. What acknowledgement gating rules out is a large and realistic class of failures: an unreachable node, a crashed node, a node that rejects a device or action it does not own, a lost command, a lost reply, and a duplicate delivery. What it does not rule out is a mechanical or electrical failure downstream of a node that is itself functioning. Closing the remaining gap requires sensing, and the incident with the unpowered servos is the strongest available evidence for why the first item of future work is position feedback rather than any software improvement.

Table 16. Objective achievement summary, assessed against the evidence of Section 7.

Obj.	Criterion	Outcome	Measured evidence
O1	Activity-level speech handled entirely offline	Met	Functional demonstration; no external service in the speech path
O2	Semantic recognition with refusal rather than guessing	Met	Weighted precision 1.00 on 110 disjoint holdout sentences; zero substitutions
O3	Environment model conditions interpretation	Met	23 location-semantics tests; workflows demonstrably differ with state
O4	One intent expands to a state-dependent sequence	Met	11 actions generated for shutdown; 14 transition-invariant tests
O5	Execution spans several independent controllers	Met	16 devices over 3 nodes; latency measured on all three
O6	State commits only on a validated correlated reply	Met	13 commit-regression and 32 emergency-life-cycle tests; containment 30/30
O7	Retry cannot cause double actuation	Met	0 re-executions in 40 replays; 4/4 with the mechanism removed
O8	State survives restart; an unclean restart is safe	Met	36 recovery and 26 persistence tests; location downgraded after a crash

Table 17. Functional requirements and their verification status.

Ref.	Requirement	Status	Verified by
FR-01	Accept natural-language voice commands	Met	Functional demonstration
FR-02	Convert voice input to text locally	Met	Functional demonstration
FR-03	Map supported commands to predefined intents	Met	Table 10
FR-04	Distinguish recognised from unrecognised commands	Met	Intent-contract tests; 0.282 hold-out rejection rate
FR-05	Require explicit human confirmation before execution	Met	20 confirmation-flow tests
FR-06	Refuse a confirmation that does not match the pending intent	Met	Confirmation-flow tests
FR-07	Maintain a representation of environment state	Met	Location and persistence suites
FR-08	Generate a multi-action workflow from one intent	Met	11-action shutdown sequence; Table 3
FR-09	Map logical devices to responsible nodes	Met	Table 6; node action-coverage tests
FR-10	Transmit commands over MQTT	Met	Table 11
FR-11	Identify commands with unique identifiers	Met	12 idempotency tests
FR-12	Wait for and correlate device status replies	Met	Commit-regression tests; Figure 8
FR-13	Retry after timeout while retries remain	Met	Table 13; retry rates 0.18 and 0.60
FR-14	Report permanent failure when attempts are exhausted	Met	Table 13; 0/9 recovery under sustained loss
FR-15	Update state only after successful execution	Met	Containment 30/30; two-tier commit tests
FR-16	Persist state and recover safely from a crash	Met	36 recovery tests
FR-17	Confirm that a physical actuator moved	Not met	Out of scope: the prototype has no sensors

7.4 Prototype versus Real-Life Deployment

The prototype is not suitable for deployment in an occupied home. This section identifies which configurations are prototype-only, what would have to change, and why the proposed change is appropriate.

Actuation: the largest gap

The SG90 servo delivers approximately 0.18 N·m of stall torque [19]. A hinged interior door leaf of realistic mass requires torque at the hinge of the order of several to tens of newton-metres to overcome inertia, latch friction and any closer, so the prototype actuator is short by roughly two orders of magnitude. No adjustment of gearing within a 9-gram plastic-gear servo closes a gap of that size; the component class is wrong, not the component.

A deployment would require a purpose-built low-energy swing-door operator: a geared direct-current motor or linear actuator with integral position sensing, a manual override permitting the door to be operated by hand when unpowered, and obstruction detection that

reverses travel on contact. Such installations are governed by safety standards for low-energy power-operated doors, which constrain opening and closing times, the force the leaf may exert and the kinetic energy it may carry, and which require the door to remain operable manually [21]. Two consequences follow for the software described in this report, and both are favourable. First, a compliant operator opens deliberately slowly, so the 700 ms settling delay would be replaced by a travel time of several seconds — which makes the five-second acknowledgement window inadequate and forces the timeout to become per-device rather than global. Second, a real operator reports position, which converts the acknowledgement from a statement about the node into a statement about the door and closes the central limitation identified in Section 7.3 at the same time.

Power

Ten SG90 servos at an approximate 650 mA stall current would demand of the order of 6.5 A if actuated simultaneously; sequential execution bounds the prototype's demand to roughly one servo at a time. A deployment using door operators cannot rely on that bound, because a real evacuation requires several doors to open at once — which is precisely the scenario in which the sequential policy produced the 13.8-fold degradation measured in Section 7.2.5. Concurrent actuation therefore becomes a requirement rather than an optimisation, and the supply must be sized for the simultaneous inrush of every actuator on the escape route, with local energy storage at each operator so that a door can complete its travel through a brief supply interruption. Battery backup sufficient to open the escape route at least once after a mains failure is a minimum for any assistive installation.

Sensing

Every reliability figure in this report is bounded by the absence of sensors. A deployment requires, at minimum, a position or limit sensor on every door, current sensing or contact feedback on every switched load, and occupancy sensing sufficient to establish the resident's location without asking them to declare it. The last of these removes the `UNKNOWN` location state's dependence on human declaration, which is the weakest link in the recovery policy: after a crash the current system simply refuses to act until the resident tells it where they are, which is an acceptable failure mode for a laboratory prototype and an unacceptable one for a person who has fallen.

Communication and security

The prototype has no authentication, no transport encryption, no device identity and no authorisation, the broker accepts anonymous connections, and the discovery beacon is unauthenticated so that any host on the subnet can redirect every node to a broker of its choosing. Wireless credentials are present in the firmware source. In an occupied home these properties mean that any device on the same network can open every door, silence the alarm or trigger the medication mechanism with a single published message. A deployment requires mutual TLS with per-device certificates, authenticated and authorised broker access, signed or authenticated discovery, credentials held outside the source tree and outside version control, and network segmentation isolating the actuation bus from general household traffic. None of this is research work; all of it is a precondition.

Availability and observability

The prototype runs a single broker and a single controller with no redundancy, performs no reconnection on the host side if the broker restarts, and has no heartbeat to detect a node

that has gone silent. The audit log grows without bound at approximately 930 bytes per committed action — 231 kB for 249 actions — because each line embeds a complete state snapshot. At a modest fifty actions per day that is roughly 17 MB per year of storage for a record that is never rotated. A deployment requires host-side reconnection with backoff, a periodic heartbeat with explicit offline marking, log rotation with a retention policy, and a structured logging framework carrying levels, timestamps, latency and retry counts in place of the prototype's unstructured console output.

Table 18. Prototype configuration against the configuration a real deployment would require, with the reason for each change.

Aspect	Prototype	Deployment requirement	Basis for the change
Door actuation	SG90 servo, approx. 0.18 N·m, 700 ms fixed settle	Low-energy swing-door operator with position feedback, manual override and obstruction reversal	Torque short by roughly two orders of magnitude; safety standards require manual operability and constrain force and kinetic energy
Acknowledgement window	5 s, global, compile-time constant	Per-device timeout derived from actuator travel time	A compliant operator opens over several seconds, exceeding the present global window
Concurrency	Strictly sequential, maximum concurrency 1	Concurrent actuation at least across independent escape doors	Measured 13.8× emergency degradation, fully explained by sequential retry waiting
Power	Shared 5 V rail; peak bounded to one servo by sequential execution	Supply sized for simultaneous inrush; local storage per operator; battery backup for the escape route	Concurrency removes the accidental bound; evacuation must survive mains loss
Sensing	None	Door position, load feedback, occupancy sensing	Six unpowered servos reported success; location certainty currently depends on human declaration
Transport security	Plaintext MQTT, anonymous broker, unauthenticated discovery	Mutual TLS, per-device identity, authorisation, authenticated discovery, network segmentation	Any subnet device can presently open every door with one message
Credentials	Wireless credentials in firmware source and in version history	Provisioned at commissioning, stored outside the source tree	Source-embedded secrets cannot be rotated and leak with the repository
Availability	Single broker, single controller, no host reconnection	Broker redundancy, host reconnection with backoff, heartbeat with offline marking	A broker restart currently leaves the host timing out until the process is restarted
Audit log	Unbounded, approx. 930 bytes per action, full snapshot per line	Rotation with retention; delta encoding; latency and retry fields	Approx. 17 MB per year at 50 actions per day, with no rotation
Recognition	Frozen encoder, 313 reference sentences, single speaker	Encoder fine-tuned on in-domain data; per-resident enrolment; multilingual coverage	0.282 holdout rejection rate; [8] reports 1–4 % rejection-AUPR gain from fine-tuning

7.5 Challenges, Limitations and Strengths

Major implementation challenges

Retry against physical actuation. The first working executor minted a fresh command identifier for each transmission attempt. This made a retry indistinguishable from a new

command at the node, and the consequence was observed directly: a door opened twice. The correction — holding the identifier stable for the logical action and suppressing duplicates in a ring buffer at the node — is the mechanism that Section 7.2.3 measures, and it was arrived at by encountering the failure rather than by anticipating it.

Deciding what an acknowledgement is permitted to prove. Considerable effort went into establishing that a reply must be correlated by identifier, device and action, and not merely be a success message arriving at the right time. Without the correlation check a reply from a different command can satisfy a wait, and the failure is entirely silent.

Representing uncertainty without corrupting the room model. An early approach treated an unknown location as another room value. Because rooms are dictionary keys in the transition tables and are matched by conditional chains with no fallback branch, this made an unknown location silently unroutable. Separating certainty onto its own axis converted a silent failure into an explicit refusal with a reason.

Simulator and firmware divergence. The software simulator and the real firmware drifted apart in two ways that the conformance tool detected: the simulator emits no reply phase field, so it cannot exercise the acknowledgement path at all, and it validates device ownership only, accepting combinations such as a television action addressed to a light that the firmware correctly refuses. Tests that pass against the simulator therefore do not fully constrain the firmware.

Technical limitations

- **No sensing of any kind.** A completion reply follows a fixed delay; movement is never confirmed. Six unpowered servos once reported success.
- **Strictly sequential execution.** Maximum concurrency is one. This produces the 13.8-fold emergency degradation and is the single largest performance limitation.
- **Timeout and retry budget are compile-time constants** that the orchestrator never overrides, so they cannot adapt to a device's actual travel time.
- **No host-side reconnection.** A broker restart leaves the application timing out on every action until the process is restarted.
- **Unbounded growth.** The reply buffer retains every status message for the process lifetime and is scanned linearly; the audit log has no rotation.
- **No logging framework.** Operational output is unstructured console printing with no levels, timestamps, latency or retry fields, so the durable record cannot answer how long an action took or how many attempts it needed.
- **Hand-written workflow builders.** Adding an intent requires editing code. Two independent intent-to-room mappings exist in the orchestrator and must be kept consistent by hand.
- **Small evaluation corpus with residual leakage.** 313 reference sentences; seven validation and four holdout sentences are near-duplicates of a reference sentence at a Jaccard similarity of 0.8 or above.
- **Untested subsystems.** The event-stream and wire-observation layer has no automated coverage, and three HTTP endpoints are asserted by no test.
- **The 55 model-marked tests are excluded from a default run,** so a passing suite has not exercised the recognition model.

Environmental and evaluation limitations

Recognition figures were obtained from written sentences and exclude transcription error entirely. All speech testing used one adult speaker, in English, in quiet conditions, with a close-talking microphone. Mechanical results were obtained on lightweight panels under no opposing load at room temperature. Long-running behaviour was never exercised. No participant from the target user population took part, so the report contains no usability evidence. The following quantities were not measured and are not estimated anywhere in this report: transcription latency, embedding inference latency, end-to-end latency from utterance to first actuation, processor and memory utilisation, power consumption, message throughput and packet loss.

Strengths of the implemented solution

- **The central safety property is enforced structurally.** One component owns state, one method is the sole execution path, and commitment is gated behind a correlated validation — all enforced by tests, including one that fails if any script bypasses the orchestrator.
- **The principal claim rests on a controlled comparison.** The idempotency result varies exactly one compile-time constant, which is a stronger form of evidence than a demonstration.
- **The recogniser fails in the safe direction.** Zero substitutions across 110 disjoint holdout sentences.
- **Negative results are reported with their magnitudes** and, in the case of the emergency degradation, with an arithmetic model that predicts the measurement to within about two per cent.
- **The verification suite is deterministic and hardware-free**, so 460 tests run anywhere and serve as a genuine regression instrument.
- **Recovery is conservative by default.** An unclean restart discards location certainty rather than assuming it, and actuators are never replayed from the audit log — which is what keeps the one non-idempotent physical action in the system safe.
- **The device abstraction proved its value under change.** Increasing the node count from two to three required edits to the registry alone, with no workflow builder altered.

8 Future Work

The improvements below are ordered by the strength of the experimental evidence that motivates them. Each is connected to a specific limitation identified during evaluation rather than proposed in the abstract.

8.1 Close the loop at the actuator

Motivated by the central limitation of Section 7.3: six unpowered servos all reported success, and every reliability figure in this report is bounded by that fact.

Fitting a position or limit sensor to every door and a current or contact sensor to every switched load would convert the completion reply from a statement about the node into a statement about the device. The firmware change is small — replace the fixed 700 ms settling delay with a wait on the sensor, bounded by a timeout, and report an explicit failure when the expected position is not reached — and the orchestrator requires no change at all, because its validation already tests the reported status rather than assuming it. This is the single modification that would most increase the value of every other result in the report, since it would allow the same experiments to be re-run against a loop that closes at the actuator. A cheaper intermediate step, available immediately, is to exploit the timing separation reported in Section 7.2.3: on slow actuators a genuine execution and a suppressed replay differ by a factor of roughly seventy-two, so a node could flag a completion that returns implausibly quickly as a suspected non-actuation without any added hardware.

8.2 Concurrent and dependency-aware execution

Motivated by the 13.8-fold emergency degradation of Section 7.2.5, whose arithmetic attributes the entire penalty to sequential waiting on retry timeouts.

Actions within a workflow are not all ordered with respect to one another. Opening the study door must precede entering the study room, but opening two independent escape doors during an evacuation has no ordering constraint whatever. Declaring dependencies explicitly in the action representation, rather than leaving them implicit in list order, would permit independent actions to be dispatched concurrently while preserving the ordering that genuinely matters. Applied to the emergency workflow this would reduce the measured worst-case duration from approximately thirty-two seconds to approximately the duration of the slowest single retry chain — a reduction of roughly an order of magnitude in exactly the scenario where latency matters most. Two changes are prerequisites: the acknowledgement gating must be extended to track several outstanding command identifiers at once, and the electrical supply must be sized for simultaneous actuation, since the present peak-current bound is an accidental consequence of sequential execution rather than a designed property.

8.3 Per-device adaptive timeouts

Motivated by the latency distribution of Section 7.2.2, where transport consumes at most 0.6 per cent of a five-second window while the medication device legitimately requires 1 902 ms.

A single global timeout must accommodate the slowest device, which makes it grossly oversized for the fastest. Deriving the window per device from its measured actuation profile — a few hundred milliseconds for a light, of the order of a second for a servo, and sev-

eral seconds for the geared door operator that a deployment would require — would shorten failure detection for fast devices by an order of magnitude without risking a premature timeout on slow ones. This becomes a necessity rather than an optimisation in deployment, because a compliant low-energy door operator opens over a period longer than the present global window.

8.4 Improve recognition recall without sacrificing precision

Motivated by the 0.282 holdout rejection rate of Section 7.2.1, and directed by the finding of Zhang *et al.* [8] that regularised fine-tuning of a sentence encoder improves rejection performance by one to four percentage points without degrading in-scope accuracy.

Lowering the threshold is the wrong lever, since it would convert refusals into substitutions and the present substitution rate on holdout is zero. The correct interventions are to enlarge and balance the reference corpus — the medication intent currently has nineteen reference sentences against fifty-eight for the emergency intent — to remove the residual near-duplicate overlap between the reference and evaluation sets, and to fine-tune the encoder on in-domain data with an out-of-scope-aware objective. A per-resident enrolment phase, in which the user contributes their own phrasings for each activity, would adapt the corpus to an individual speaker and is likely to yield a larger improvement than any change to the matching rule.

8.5 End-to-end latency instrumentation

Motivated by the gap acknowledged in Section 7.5: transcription, embedding and utterance-to-first-actuation latency were never measured, so the report can characterise the execution path precisely and the interpretation path not at all.

Instrumenting the speech and recognition stages with the same monotonic timing used for execution would complete the latency picture and establish whether the interpretation stages, which run substantial models on a CPU, dominate the user-perceived response time. Given that the entire execution path completes in well under a second for a single-room workflow, it is probable that they do, and that conclusion would redirect future optimisation effort away from the transport entirely.

8.6 Operational robustness

Motivated by the limitations listed in Section 7.5.

Four changes would move the system from a bench prototype toward something that could run unattended: host-side broker reconnection with exponential backoff, so that a broker restart does not leave the application timing out indefinitely; a periodic node heartbeat with explicit offline marking, so that the loss of a node is surfaced immediately rather than discovered when the resident asks for something in that region of the home; bounded retention for the reply buffer and rotation for the audit log, with delta encoding in place of the present full-snapshot-per-line format; and a structured logging framework carrying levels, timestamps, per-action latency and retry counts, which would allow every measurement reported in Section 7.2 to be recomputed from the durable record instead of requiring a bespoke instrument.

8.7 Security

Motivated by the deployment analysis of Section 7.4.

Mutual TLS with per-device certificates, authenticated and authorised broker access, authentication of the discovery beacon, removal of wireless credentials from the firmware source, and segmentation of the actuation bus from general household traffic are preconditions for any installation in an occupied home. None constitutes research, and all should be regarded as work that must precede rather than accompany further capability development.

8.8 Evaluation with the target population

No participant with a mobility impairment took part in this project, so it contains no evidence about usability or acceptability. The field study of Vigouroux *et al.* [1] indicates that command formulation, intelligibility and speech segmentation dominate real-world failure in this population — categories that a written-sentence evaluation cannot observe. A study with representative users, measuring end-to-end spoken success rather than text-level intent matching, is required before any claim about assistive effectiveness can be made.

9 Conclusion

This project addressed the problem of converting a spoken activity request into a reliable multi-device operation in an assistive environment built from actuators that provide no feedback. The difficulty is twofold: an activity such as preparing for sleep or evacuating a home is an ordered sequence whose content depends on where the resident is and on what is already switched on, and a controller that records the intended effect of a published command will accumulate a false picture of the home whenever an actuator fails silently.

The implemented solution is a six-layer orchestration framework running entirely on local hardware. Speech is captured at 16 kHz and transcribed offline by faster-whisper. The transcript is embedded with a frozen sentence encoder and scored by maximum cosine similarity against a 313-sentence reference corpus, with a single threshold of 0.65 separating a predicted intent from an explicit refusal. A recognised intent is executed only after the resident confirms it. Two policy gates then apply, a state-conditioned builder expands the intent into an ordered action list, and each action is dispatched individually over MQTT at QoS 1 to one of three ESP32 nodes driving five lights, ten servos and one buzzer. The logical effect of an action is admitted into the environment model only after a reply has arrived carrying the dispatched command identifier, naming the same device and action, and reporting success. Device facts are committed as each acknowledgement arrives; the resident's room and the operating mode are committed only after the whole sequence has run. A retry reuses the original command identifier and each node suppresses duplicates against a thirty-two-entry ring buffer. State is persisted atomically, audited append-only, and restored under a recovery policy that discards location certainty after an unclean shutdown rather than assuming it.

The major achievements are these. A single intent expands into a state-dependent sequence of up to eleven actions spanning three independent controllers. Identifier-stable retry with node-side duplicate suppression eliminated duplicate physical actuation entirely: forty replays across ten devices produced zero re-executions, while the same procedure with the mechanism compiled out re-executed on four replays out of four for every servo tested. Intent recognition on a disjoint, balanced 110-sentence holdout set achieved 0.718 accuracy with a weighted precision of 1.00 across all eleven intents, meaning that every recognition error was a refusal and none was a substitution of one activity for another. Transient node loss was recovered in six of six cases, sustained loss in none as the bounded retry budget intends, and failure containment after an abort was exact at thirty of thirty. A deterministic suite of 460 hardware-free tests verifies the commit rule, the emergency life-cycle, the recovery policy and the transition invariants.

Two experimental findings deserve particular emphasis because they concern the design's costs rather than its benefits. Acknowledgement gating is close to free: transport occupies 13 to 30 ms against actuation costs of 56 to 1 902 ms, so waiting for a correlated reply adds no meaningful latency at this scale. Conversely, the combination of a best-effort emergency policy with strictly sequential execution degrades badly under partial failure — losing two devices of six increased the median emergency duration from 2 335 ms to 32 256 ms, a factor of 13.8 — and a simple arithmetic model built from the retry budget predicts that measurement to within about two per cent, which identifies precisely which two constants would have to change.

All eight objectives stated in Section 2.4 were met, two of them in a modified form justified in Section 4.2. One capability discussed throughout was not delivered and is not claimed: the prototype contains no sensors, so a completion reply establishes that a node executed its actuation routine, not that a door moved. The incident in which six unpowered servos all reported success defines that boundary exactly, and it is the reason position feedback heads the future work.

The practical significance of the project lies in what the measurements show about the cost of doing this correctly. Verification at the point of actuation is frequently omitted from low-cost assistive systems on the assumption that it is expensive; the latency figures reported here indicate that on hardware of this class it is not, and the idempotency result shows that the omission carries a concrete hazard — a door that opens twice because an acknowledgement was lost. For a resident who depends on that door, the difference between a controller that knows what it has achieved and one that assumes it is not an implementation detail. It is the property that makes the system trustworthy, and this report demonstrates that the property can be obtained on commodity microcontrollers at negligible cost in time.

10 References

- [1] N. Vigouroux, F. Vella, G. Lepage, and É. Campo, "Design recommendations based on speech analysis for disability-friendly interfaces for the control of a home automation environment," arXiv preprint arXiv:2311.13223, Nov. 2023.
- [2] C. Irugalbandara, A. S. Naseem, S. Perera, S. Kiruthikan, and V. Logeeshan, "A secure and smart home automation system with speech recognition and power measurement capabilities," *Sensors*, vol. 23, no. 13, art. 5784, Jun. 2023, doi: 10.3390/s23135784.
- [3] A. Brunete, E. Gambao, M. Hernando, and R. Cedazo, "Smart assistive architecture for the integration of IoT devices, robotic systems, and multimodal interfaces in healthcare environments," *Sensors*, vol. 21, no. 6, art. 2212, Mar. 2021, doi: 10.3390/s21062212.
- [4] A. Katumba, S. Murindanyi, J. T. Kasule, and E. Mugume, "Luganda speech intent recognition for IoT applications," arXiv preprint arXiv:2405.19343, May 2024.
- [5] S. Dutta, S. Chandupatla, and J. H. L. Hansen, "Adapting Whisper for lightweight and efficient automatic speech recognition of children for on-device edge applications," arXiv preprint arXiv: 2507.14451, Jul. 2025.
- [6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, Nov. 2019, pp. 3982–3992.
- [7] S. Larson, A. Mahendran, J. J. Peper, C. Clarke, A. Lee, P. Hill, J. K. Kummerfeld, K. Leach, M. A. Laurenzano, L. Tang, and J. Mars, "An evaluation dataset for intent classification and out-of-scope prediction," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, Nov. 2019, pp. 1311–1316.
- [8] T. Zhang, A. Norouzzian, A. Mohan, and F. Ducatelle, "A new approach for fine-tuning sentence transformers for intent classification and out-of-scope detection tasks," arXiv preprint arXiv: 2410.13649, Oct. 2024.
- [9] E. King, H. Yu, S. Lee, and C. Julien, "Sasha: Creative goal-oriented reasoning in smart homes with large language models," arXiv preprint arXiv:2305.09802, May 2023.
- [10] D. Rivkin, F. Hogan, A. Feriani, A. Konar, A. Sigal, S. Liu, and G. Dudek, "SAGE: Smart home agent with grounded execution," arXiv preprint arXiv:2311.00772, Nov. 2023.
- [11] C. Yu, C. Choi, S. Lee, H. Kim, S. Y. Ko, Y.-B. Ko, and S. Oh, "Leveraging LLMs for efficient and personalized smart home automation," arXiv preprint arXiv:2601.04680, Jan. 2026.
- [12] Z. Zhan, K. Li, Y. Zhang, and H. Haddadi, "HearthNet: Edge multi-agent orchestration for smart homes," arXiv preprint arXiv:2604.09618, Mar. 2026.
- [13] Y. Du, Q. Yang, L. Wang, J. Lin, H. Cui, and S. C. Liew, "LLMind 2.0: Distributed IoT automation with natural language M2M communication and lightweight LLM agents," arXiv preprint arXiv: 2508.13920, Aug. 2025.
- [14] Y. Cheng, M. Xu, Y. Zhang, K. Li, R. Wang, and L. Yang, "AutoIoT: Automated IoT platform using large language models," arXiv preprint arXiv:2411.10665, Nov. 2024.
- [15] G. Seo, J. Yang, J. Pyo, N. Kim, J. Lee, and Y. Jo, "SimuHome: A temporal- and environment-aware benchmark for smart home LLM agents," arXiv preprint arXiv:2509.24282, Sep. 2025.
- [16] J. Ali, M. H. Zafar, C. Hewage, R. Hassan, and R. Asif, "Mathematical modeling and validation of retransmission-based mutant MQTT for improving quality of service in developing smart cities," *Sensors*, vol. 22, no. 24, art. 9751, Dec. 2022, doi: 10.3390/s22249751.
- [17] A. Banks and R. Gupta, Eds., *MQTT Version 3.1.1*, OASIS Standard, Oct. 2014.

-
- [18]** Espressif Systems, *ESP32-WROOM-32 Datasheet*, Espressif Systems (Shanghai) Co., Ltd.
- [19]** TowerPro, *SG90 9 g Micro Servo — Technical Specification*, TowerPro Pte. Ltd.
- [20]** A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, Honolulu, HI, USA, 2023, pp. 28492–28518.
- [21]** Builders Hardware Manufacturers Association, *ANSI/BHMA A156.19 — Power Assist and Low Energy Power Operated Doors*, BHMA, New York, NY, USA.
- [22]** Eclipse Foundation, *Eclipse Mosquitto — An Open Source MQTT Broker*, Eclipse Foundation.

11 Appendix

A Message schemas on the MQTT bus

Two topic trees carry all application traffic. Commands are published by the host to `assistive/command/<node_id>`; replies are published by the node to `assistive/status/<node_id>`. Both are JSON and both are published at QoS 1.

```
// command, host → node
{
  "device":      "sleep_door",      logical device identifier, never a pin
  "action":      "OPEN_DOOR",      member of the 13-type action vocabulary
  "parameters": { },              optional; empty for most actions
  "command_id":  "4f1c..."        UUID, stable across every retry attempt
}

// reply, node → host – phase "ack", published on receipt
{ "command_id": "4f1c...", "node": "esp32_b", "phase": "ack" }

// reply, node → host – phase "complete", published after the actuation routine
{
  "command_id": "4f1c...",
  "node":      "esp32_b",
  "device":    "sleep_door",
  "action":    "OPEN_DOOR",
  "status":    "success",          "success" | "error"
  "phase":    "complete",
  "duplicate": false              true when answered from the ring buffer
}
```

The executor waits specifically for a reply whose phase is `complete` and whose command identifier matches the one dispatched. The orchestrator applies a further check before committing anything, requiring the device and action fields to match the action that was sent and the status to be `success`.

B Discovery beacon payload

```
// UDP broadcast on port 18830, every 2 s, to 255.255.255.255 and the subnet address
{ "service": "assistive-broker", "broker": "192.168.x.x", "port": 1883 }
```

A node listens for this beacon only after three consecutive failures to connect to its last known broker address, which prevents the discovery mechanism from interfering with normal operation. The beacon is unauthenticated; the consequences are stated in Section 7.4.

C Firmware constants governing the measured behaviour

Table 19. Firmware and host constants that determine the timing and idempotency results of Section 7.2. All are compile-time values in the delivered build.

Constant	Value	Effect on measured behaviour
Servo settling delay	700 ms	Dominates ack-to-complete for every servo; the source of the 691–704 ms cluster in Table 11
Completed-command ring size	32 entries	Bounds how many past command identifiers a node can recognise as duplicates
Acknowledgement window	5 s	With the retry budget, produces the 30 s of pure waiting in the degraded emergency case
Retry budget	2 retries (3 attempts)	Determines the boundary between a recovered transient loss and an abandoned device
Similarity threshold	0.65	Separates PREDICTED from UNKNOWN; sets the 0.282 hold-out rejection rate
Reserved pins checked at boot	GPIO 2, 25	Excluded from device assignment on the module variant used
Beacon interval / failure threshold	2 s / 3	Governs how quickly a node re-acquires a broker that has changed address

D Evaluation datasets

Table 20. Composition of the three datasets. Exact overlap between the reference corpus and either evaluation set is zero.

Dataset	Sentences	Per intent	Role
Reference corpus	313	19 – 58	Retrieval set the recogniser matches against; not trained on
Validation	105	8 – 10	Secondary evaluation; 7 sentences are near-duplicates of a reference sentence
Holdout	110	10 (balanced)	Primary evaluation; 4 sentences are near-duplicates of a reference sentence

E Reproducing the reported results

The deterministic verification suite requires no broker, no hardware, no microphone and no model download, and redirects its state directory per test. The hardware measurements require the three nodes, the broker and the bench instruments described in Section 6.3. The idempotency comparison additionally requires a second firmware build in which the duplicate ring buffer is compiled out; that build prints a warning at start-up and exists only for this experiment.

OPTIONAL ADDITIONS TO THIS APPENDIX

Depending on departmental preference, the appendix may also carry: the complete reference-corpus listing; the full firmware sketch; per-intent precision, recall and F1 tables from the recognition evaluation; and the raw per-trial timing records behind Tables 11 to 14. The guidelines require that the appendix not be used to relocate essential explanation out of the main report, so none of the analysis above depends on this material.